

School of Engineering
MSc(Eng) Project

Open-Architecture Mini Robotic Arm Development for Pharmaceutical Industry 4.0

Kieran Pettigrew
Advanced Aerospace Engineering MSc
August 2026

Supervisor:
Hay Wong
Project No. 123

School of Engineering, University of Liverpool
Brownlow Hill, Liverpool, L69 3GH, United Kingdom

Abstract

This project presents the design, manufacture, and evaluation of a low-cost robotic inspection and sorting system developed as a proof-of-concept Industry 4.0 application. The system combines a four-degree-of-freedom serial manipulator, a separately actuated gripper, five servo actuators driven through a PCA9685 PWM controller, an OV7670 camera, Arduino-based hardware control, and Python software for machine vision, trajectory generation, and inverse kinematics. Coloured demonstration vials are identified at three predefined positions using region-of-interest sampling and HSV-based colour classification, after which the control system autonomously performs the required pick-and-place sorting sequence.

The manipulator was manufactured primarily from laser-cut plywood and developed iteratively to address issues including joint friction, gripper mass, actuator loading, backlash, and mechanical alignment. A closed-form inverse-kinematic model was used to calculate joint configurations, with servo calibration, joint limits, and intermediate Cartesian waypoints incorporated to improve practical operation.

Experimental testing showed a mean absolute joint-position error of 0.58° and a maximum error of 2°. The vision system correctly classified the vial arrangement in all 18 end-to-end trials, while the complete automated sorting sequence was successfully completed in 16 trials, giving an overall success rate of 88.9%. The two unsuccessful trials resulted from small positioning errors during vial pick-up.

The results demonstrate that low-cost hardware can successfully integrate sensing, automated decision-making, kinematic control, and robotic manipulation within a constrained proof-of-concept system, while highlighting limitations in positional accuracy, repeatability, and environmental robustness compared with industrial robotic systems.

Contents

Abstract **i**

1 Introduction **1**
1.1 Background . . . 1
1.2 Aim and Objectives . . . 1
1.3 Sustainability and UN Sustainable Development Goals . . . 2

2 Literature Review **3**
2.1 Industry 4.0 and Robotics . . . 3
2.2 Robotic Arms for Inspection and Sorting . . . 3
2.3 Vision and Sensing for Object Detection . . . 4
2.4 Gripper Design . . . 4
2.5 Low-Cost Robotic Prototyping . . . 5
2.6 Kinematics and Control of Robotic Arms . . . 5
2.6.1 Forward Kinematics . . . 5
2.6.2 Inverse Kinematics . . . 7
2.7 Literature Review Summary . . . 9

3 System Requirements and Conceptual Design **10**
3.1 Requirements . . . 10
3.2 Concept Development . . . 11
3.3 Final System Architecture . . . 11
3.4 Industry 4.0 Relevance . . . 11
3.5 Repurposing and Modular Design Principles . . . 11

4 Mechanical Design **13**
4.1 Manipulator Configuration and Geometry . . . 13
4.2 Joint Design . . . 14
4.2.1 Base Joint . . . 14
4.2.2 Shoulder Joint . . . 15
4.2.3 Elbow Joint . . . 16
4.2.4 Wrist Joint . . . 16
4.3 Gripper Design . . . 17
4.4 CAD Development . . . 17
4.5 Laser-Cutter Manufacturing . . . 18
4.6 Mechanical Design Iterations . . . 19

5 Control System **21**
5.1 System Architecture . . . 21
5.2 Control Hardware . . . 22
5.2.1 Arduino UNO R3 . . . 22
5.2.2 PCA9685 PWM Driver . . . 22

5.2.3 Servo Actuation . . . 22
5.2.4 Power Arrangement . . . 23
5.2.5 OV7670 Camera Interface . . . 23
5.2.6 Wiring . . . 24
5.3 Kinematics and Manipulator Control Software . . . 24
5.3.1 Coordinate Systems . . . 25
5.3.2 Forward Kinematics . . . 25
5.3.3 Inverse Kinematics . . . 27
5.3.4 Joint Limits . . . 29
5.3.5 Trajectory Generation . . . 31
5.3.6 Motion Smoothing . . . 32
5.3.7 Servo Calibration . . . 33
5.3.8 Safety and Emergency Stop . . . 34
5.3.9 Robot Simulation . . . 34
5.4 Machine Vision and Colour Classification . . . 35
5.4.1 Image Acquisition . . . 36
5.4.2 Region-of-Interest Colour Classification . . . 36
5.4.3 Sorting Logic . . . 37
5.4.4 Vision System Limitations . . . 37
5.5 Integration Challenges . . . 38
5.5.1 Camera Hardware Fault . . . 38
5.5.2 Communication and Timing . . . 38
5.5.3 Kinematic Model and Physical Servo Calibration . . . 38
5.5.4 Trajectory and Actuator Loading . . . 39
5.5.5 Gripper and Motion Sequencing . . . 39

6 Testing and Results **40**
6.1 Testing Methodology . . . 40
6.2 Joint Position Testing . . . 40
6.3 End-to-End Automated Sorting Test . . . 41

7 Discussion **43**
7.1 Achievement of Project Objectives . . . 43
7.2 Mechanical Performance . . . 43
7.3 Control System Performance . . . 44
7.4 Vision System Performance . . . 45
7.5 Industry 4.0 Application . . . 45
7.6 System Limitations . . . 46

8 Conclusions and Future Work **47**
8.1 Conclusions . . . 47
8.2 Future Work . . . 47

References **49**

A Mechanical Drawings **51**

List of Notation

Symbol	Description	Unit
x, y, z	Cartesian coordinates of the end-effector	mm
h	Fixed height from the robot base to the shoulder joint	mm
L_1	Length of the first arm link	mm
L_2	Length of the second arm link	mm
L_3	Effective length from the wrist joint to the gripper end	mm
θ_1	Base joint angle	$^\circ$
θ_2	Shoulder joint angle	$^\circ$
θ_3	Elbow joint angle	$^\circ$
θ_4	Wrist joint angle	$^\circ$
θ_g	Gripper actuator angle	$^\circ$
ϕ	Desired end-effector orientation within the arm plane	$^\circ$
r_4	Horizontal radial distance from the shoulder joint to the end-effector	mm
h_4	Vertical distance from the shoulder joint to the end-effector	mm
r_3	Horizontal radial distance from the shoulder joint to the wrist joint	mm
h_3	Vertical distance from the shoulder joint to the wrist joint	mm
d	Distance from the shoulder joint to the required wrist joint	mm
P	Cartesian position vector	mm
P_0	Start position of a Cartesian trajectory	mm
P_1	End position of a Cartesian trajectory	mm
P_i	i-th interpolated Cartesian trajectory point	mm
s_i	Normalised interpolation parameter at trajectory step i	–
i	Trajectory interpolation step index	–
N	Number of trajectory interpolation intervals	–
Δt	Time interval between successive motion updates	ms
ω_{arm}	Programmed angular speed of the manipulator joints	$^\circ$/s
ω_{gripper}	Programmed angular speed of the gripper	$^\circ$/s
$\theta_{\min}$	Minimum permitted joint angle	$^\circ$
$\theta_{\max}$	Maximum permitted joint angle	$^\circ$
θ_{target}	Requested target angle of a joint	$^\circ$
θ_k	Commanded joint angle at motion-control step k	$^\circ$
R, G, B	Red, green and blue image intensity components	–
H, S, V	Hue, saturation and value colour components	–

List of Figures

4.1 Full robot arm assembly. 13
4.2 Manipulator configuration diagram. 14
4.3 Deconstructed view of the base unit. (1) Ball bearings sitting in base tracks. (2) Upper bearing track. (3) Base servo mount. (4) Base clamping ring, secures the arm down onto the bearings. 15
4.4 The shoulder joint. (1) Base clamp ring. (2) Shoulder servo mount. (3) Servo to arm actuation interface. (4) Shoulder structural support. 15
4.5 The elbow joint. (1) Custom servo actuation arm. (2) Carbon fibre actuation pin. (3) Elbow servo mounts. 16
4.6 The wrist joint using fundamentally the same mechanism as the elbow. . . 16
4.7 The gripper and its operating mechanism. (1) Gripper servo. (2) Servo actuated gripper linkage. (3) Second linkage actuated by gear. (4) Nyloc-nuts. (5) Curved finger ribs. 17
4.8 CREO Parametric CAD model of the complete robotic arm assembly. . . . 18
4.9 Custom-designed laser-cutter tolerance tester. 19

5.1 Functional architecture and control-data flow of the completed robotic inspection and sorting system. 21
5.2 Wiring diagram of the robotic arm control system, produced in Cirkit Designer (Cirkit Designer 2026). For online view follow: `https://app.cirkitdesigner.com/project/0900e36c-803b-4681-b52b-90e4940d26d2` 24
5.3 Diagram detailing how the code handles link angles relative to previous frame orientation. 25
5.4 Three-dimensional Matplotlib visualisation of the robotic arm. 35
5.5 Machine-vision classification using the predefined regions of interest. 36

List of Tables

3.1 System requirements of the robotic sorting system. 10

5.1 PCA9685 servo channel allocation. 22
5.4 Joint limits. 30
5.5 Hue classification ranges. 37

6.1 Joint position accuracy results. 41
6.2 End-to-end automated sorting test results. 42

1. Introduction

This chapter introduces the industrial context and motivation for the development of the robotic sorting system, defines the project aims and objectives, and identifies the prototype's sustainability relevance.

1.1 Background

Industrial automation has become increasingly important within modern manufacturing as organisations seek to improve productivity, consistency, quality, and workplace safety. Robotic systems are particularly well suited to tasks that are repetitive, require consistent positioning, or involve the handling and inspection of large numbers of similar products. Industrial robots are therefore widely used for applications including assembly, material handling, inspection, packaging, and sorting.

Low-cost robotic platforms offer a practical way to investigate these principles at proof-of-concept scale without attempting to match the accuracy or reliability of commercial industrial systems.

This project investigated the development of a low-cost robotic vision based sorting system as a proof-of-concept Industry 4.0 application. The system used a four-degree-of-freedom robotic manipulator with a separately actuated gripper to handle coloured demonstration vials. A fixed camera identified the colour of vials at predefined inspection positions, after which the control system determined whether each vial should remain in position or be transported to one of two sorting locations.

The project emphasised the integration of the mechanical, electronic, and software subsystems, including manipulator and gripper design, forward and inverse kinematics, servo control, machine vision, and autonomous execution of the complete inspection and sorting sequence.

1.2 Aim and Objectives

The overall aim of this project was to design, manufacture, and integrate a low-cost robotic system capable of automatically identifying and sorting coloured demonstration vials as a proof-of-concept Industry 4.0 application.

To achieve this aim, the following objectives were defined:

1. To design and manufacture, within the project timescale, a four-degree-of-freedom robotic manipulator with a separately actuated gripper capable of reaching all predefined vial pick-up and sorting positions.
2. To develop and integrate the kinematic, control, and machine-vision systems required for the robot to identify red, green, and blue vials and autonomously exe-

cute the corresponding pick-and-place sorting sequence without continuous manual control.

3. To evaluate the completed prototype through joint-position accuracy testing and repeated end-to-end sorting trials, recording the success rate of the complete automated sequence and identifying any recurring failure modes.

1.3 Sustainability and UN Sustainable Development Goals

The project is relevant to the Sustainable Development Goal 9, *Industry, Innovation and Infrastructure* , through its investigation of accessible automation technologies for modern manufacturing (United Nations 2026). Low-cost robotic systems can provide a platform for developing and evaluating automation concepts without the financial investment required for industrial equipment, which may be useful in education, small organisations, and early-stage technology development.

Automated inspection and sorting can also support improved process control and reduced waste, thereby providing a secondary link to Sustainable Development Goal 12, *Responsible Consumption and Production* (United Nations 2026). The present prototype demonstrates only a simplified colour-based sorting task, so no direct material or energy saving benefits were measured.

The prototype itself was also developed using relatively low-cost components and laser-cut plywood structures that could be modified and re-manufactured rapidly during development. While this approach was primarily selected for cost and ease of manufacture, it also reduced the need for more wasteful material-intensive machining processes during iterative prototyping.

2. Literature Review

The literature review examines the engineering principles required for a low-cost robotic inspection and sorting system: Industry 4.0 automation, pick-and-place robotics, sensing and colour classification, manipulator kinematics, gripper design, low-cost prototyping, and the additional challenges of conveyor-based sorting.

2.1 Industry 4.0 and Robotics

Industry 4.0 describes the increasing integration of physical manufacturing systems with digital technologies, communication networks, sensing, and automated decision-making (Koh, Orzes, and Jia 2019). It developed from earlier stages of industrial automation by extending computer-controlled manufacturing into interconnected cyber-physical systems capable of collecting, exchanging, and acting upon process information. Technologies commonly associated with Industry 4.0 include robotics, machine vision, artificial intelligence, sensors, and the Industrial Internet of Things (IoT) (Skilton and Hovsepian 2018).

Robotics has an important role within this environment since machines can carry out repetitive, hazardous, and precision tasks with greater consistency than manual labour. Industrial robots are widely used for tasks including assembly, material handling, inspection and sorting (Soori et al. 2024). Their capability can be extended through integration with sensing and intelligent control, allowing robotic systems to respond to information obtained from the production environment rather than performing only fixed sequences of motion (Singh 2025).

The combination of sensing, automated decision-making, and robotic manipulation is therefore particularly relevant to the present project. Rather than operating solely as a programmed manipulator, the developed system uses visual information to determine the required physical handling operation, demonstrating these Industry 4.0 principles at proof-of-concept scale.

2.2 Robotic Arms for Inspection and Sorting

Robotic arms are commonly used for pick-and-place operations in which an object is grasped, transported, and released at another location. Such systems are widely applied to manufacturing tasks including packaging, assembly, material handling, and sorting (Surati et al. 2021). For inspection and sorting applications, sensing can first be used to classify a product according to characteristics such as colour, geometry, or identity before the robot performs the required handling action (Zou et al. 2021). Compared with manual handling, automation can improve consistency and repeatability while reducing repetitive human labour and exposure to hazardous processes (Trevelyan, Hamel, and Kang 2016; Ajiga et al. 2024).

Effective implementation requires the sensing, classification, and manipulation processes to operate as an integrated system. The manipulator must provide sufficient workspace

and positioning capability to reach the objects and sorting destinations, while the end-effector must reliably handle the selected product. The inspection system must then provide sufficient information for the controller to determine the required response. Inspection and sorting are particularly suited to automation because consistent product classification can reduce errors, rework, and the risk of unsuitable products progressing through a manufacturing process (Magalhaes and Ferreira 2022).

Industrial sorting systems are frequently combined with conveyors so that products can be inspected and removed while travelling through a production line (Abbood, Abdullah, and Khalid 2019). However, dynamic conveyor-based manipulation introduces additional requirements because the object position changes continuously, requiring accurate timing, synchronisation, and faster sensing and motion control. Static sorting therefore provides a simpler method of demonstrating the fundamental inspection and manipulation process before extending the system to moving products. For this reason, the present project concentrates on vials held at predefined positions, while conveyor-based operation represents a possible future development.

2.3 Vision and Sensing for Object Detection

Automated inspection systems can obtain information using sensors including cameras, colour sensors, proximity sensors, ultrasonic sensors, and barcode readers (De et al. 2025). The appropriate sensing method depends on the information required from the object. Simple sensors can provide highly specific measurements, whereas camera-based machine vision can obtain multiple characteristics, including colour, position, shape, and orientation, without physical contact (Malburg et al. 2021). This makes machine vision particularly useful for automated inspection and sorting.

Colour provides a relatively simple basis for classification where objects are intentionally distinguishable by their appearance. Colour information obtained from a camera can be analysed using RGB or alternative colour representations to allocate objects to predefined categories (Shen and Hassan 2014). For a low-cost sorting prototype, this avoids the computational complexity associated with general-purpose object recognition while still demonstrating the integration of visual sensing and automated decision-making.

The principal limitation of camera-based colour classification is its sensitivity to the imaging environment. Changes in illumination, shadows, reflections, and background conditions can alter the measured colour values and reduce classification reliability. Cameras also require additional image acquisition and processing compared with dedicated colour sensors. These limitations favour a controlled inspection area, fixed camera geometry, and consistent object positioning where practical.

2.4 Gripper Design

The gripper forms the interface between the robotic manipulator and the handled object and must provide sufficient constraint to allow reliable pickup, transport, and release. Inappropriate gripper geometry or insufficient contact can result in slipping, misalignment, or damage to the object (Hernandez et al. 2023). Gripper selection must therefore consider the geometry, mass, and surface characteristics of the intended payload.

Common robotic end-effectors include mechanical finger grippers, suction devices, magnetic grippers, soft grippers, and application-specific mechanisms (Hernandez et al. 2023). Mechanical two-finger grippers are particularly suitable for relatively small objects because they are simple to manufacture and actuate. Cylindrical objects present an additional challenge because they can rotate or slip between flat contact surfaces, making shaped or conforming contact geometry advantageous (Borkar et al. 2023). For the present application, these considerations support the use of a custom two-finger gripper shaped specifically around the demonstration vial.

2.5 Low-Cost Robotic Prototyping

Low-cost development platforms make it possible to investigate robotic control and automation without the expense of industrial hardware. Arduino is widely used in educational and prototype robotics because it provides straightforward interfaces for sensors, actuators, and serial communication while supporting rapid development (Kapoor 2023). In a system such as this, higher-level processing can be carried out on a host computer while the microcontroller provides the interface to the physical hardware.

Rapid-prototyping methods such as additive manufacturing and laser cutting are also well suited to low-cost robotic development because custom components can be produced directly from CAD models and modified quickly when design problems are identified (Protolabs 2024; Marks 2019). This supports an iterative development process for links, brackets, grippers, and other structural components.

The main disadvantage of low-cost robotic systems is reduced performance compared with industrial manipulators. Limitations can include lower structural stiffness, actuator torque, positional accuracy, repeatability, and payload capacity, together with effects such as servo backlash (Salazar et al. 2024). The present system should therefore be considered a proof-of-concept platform for demonstrating robotic manipulation, sensing, and automated sorting rather than a replacement for industrial equipment.

2.6 Kinematics and Control of Robotic Arms

Kinematics describes the motion and geometry of mechanical systems without considering the forces and moments responsible for that motion. For robotic manipulators, kinematic analysis establishes the relationship between the configuration of the robot's joints and the position and orientation of its end-effector (Craig 2005). This relationship forms the basis of both forward and inverse manipulator kinematics.

2.6.1 Forward Kinematics

Forward kinematics determines the position and orientation of a manipulator's end-effector from known joint variables and fixed geometric parameters. For a serial manipulator, the individual links are connected sequentially, meaning that the pose of the end-effector depends on the accumulated transformations between each successive link. The forward kinematic problem can therefore be regarded as a mapping from joint space to Cartesian space (Craig 2005).

A rigid-body pose requires both position and orientation to be represented. Craig describes this using coordinate frames attached to rigid bodies, with the orientation of one frame relative to another represented by a rotation matrix and its displacement represented by a position vector. These can be combined into a homogeneous transformation matrix of the form

$$ {}^{A}T_{B} = \begin{bmatrix} {}^{A}R_{B} & {}^{A}P_{B} \\ 0 \quad 0 & 0 \quad 1 \end{bmatrix}, \tag{2.1} $$

The rotation matrix ${}^{A}R_{B}$ contains the orientation of the three axes of frame $\{B\}$ expressed relative to frame $\{A\}$:

$$ {}^{A}R_{B} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}, \tag{2.2} $$

where the three columns represent the directions of the x, y and z axes of frame $\{B\}$ expressed in frame $\{A\}$.

The position vector ${}^{A}P_{B}$ contains the Cartesian coordinates of the origin of frame $\{B\}$ expressed in frame $\{A\}$:

$$ {}^{A}P_{B} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}. \tag{2.3} $$

Together, these quantities describe the complete pose of frame $\{B\}$ relative to frame $\{A\}$. Homogeneous transformation matrices are therefore useful because they allow both rotation and translation to be represented within a single matrix and combined through matrix multiplication (Craig 2005).

For an n-link serial manipulator, coordinate frames may be attached to successive links so that the transformation between neighbouring frames is represented by ${}^{i-1}T_{i}$. The transformation between the base and end-effector frames is consequently

$$ {}^{0}T_{n} = {}^{0}T_{1}\,{}^{1}T_{2} \cdots {}^{n-1}T_{n}. \tag{2.4} $$

The final matrix therefore contains both the position and orientation of the end-effector relative to the base frame.

Denavit–Hartenberg Convention

A general serial manipulator can contain joint axes with different positions and orientations, making the consistent assignment of coordinate frames important. The Denavit–Hartenberg (D–H) convention provides a systematic method of attaching frames to successive links and describing the relative geometry using four parameters per link (Craig 2005). Craig's convention attaches frame $\{i\}$ to link i, with its z_i-axis aligned with joint

axis i. The x_i-axis is normally chosen along the common perpendicular between successive joint axes, with the remaining axis completing a right-handed coordinate frame.

The relative geometry of neighbouring links can then be described using four D–H parameters:

- a_{i-1} – the link length, representing the distance between successive joint axes measured along x_{i-1};
- α_{i-1} – the link twist, representing the angle between successive joint axes measured about x_{i-1};
- d_i – the link offset, representing the displacement along the joint axis z_i;
- θ_i – the joint angle, representing rotation about the joint axis z_i.

For a revolute joint, θ_i is the joint variable while the remaining parameters are constant. For a prismatic joint, d_i becomes the joint variable instead (Craig 2005). Craig notes that these four quantities provide a general kinematic description of the relationship between adjacent links.

Using these parameters, the transformation between neighbouring link frames may be written as

$$
{}^{i-1}T_i = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -d_i s\alpha_{i-1} \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & d_i c\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{2.5}
$$

where c and s denote cosine and sine respectively. Successive D–H transformations can then be multiplied to obtain the complete forward kinematic model of a manipulator. The importance of the D–H convention is therefore not that it changes the underlying geometry, but that it provides a systematic and repeatable method for describing potentially complex serial mechanisms.

2.6.2 Inverse Kinematics

Inverse kinematics addresses the opposite problem to forward kinematics. Given a required end-effector position and orientation, the joint variables required to produce that pose must be determined (Craig 2005; Mohammed and Sunar 2015). Whereas forward kinematics maps known joint variables to a single end-effector pose, inverse kinematics begins with the desired Cartesian pose and determines the corresponding joint configuration.

For a manipulator with joint variables $\theta_1, \theta_2, \ldots, \theta_n$, the forward kinematic relationship may be written generally as

$$
{}^0T_n = f(\theta_1, \theta_2, \ldots, \theta_n), \tag{2.6}
$$

where ${}^0T_n$ represents the pose of the end-effector relative to the base frame. The inverse kinematic problem is therefore to determine values of

$$\theta_1, \theta_2, \ldots, \theta_n \tag{2.7}$$

that produce a specified end-effector transformation ${}^0T_n$.

Although inverse kinematics can be viewed conceptually as the reverse of forward kinematics, it is generally more difficult because the resulting equations are nonlinear and may not have a unique solution. Craig identifies three principal issues in inverse manipulator kinematics: whether a solution exists, whether multiple solutions exist, and the method used to obtain those solutions (Craig 2005).

Existence of Solutions and Workspace

An inverse-kinematic solution exists only if the requested pose lies within the workspace of the manipulator. Craig distinguishes between the *reachable workspace*, containing points that the end-effector can reach in at least one orientation, and the *dexterous workspace*, containing points that can be reached with all required orientations (Craig 2005). The dexterous workspace is therefore generally smaller than the reachable workspace.

The practical workspace can be reduced further by mechanical joint limits. Consequently, a Cartesian position may be geometrically reachable but still require a joint configuration that cannot be achieved by the physical manipulator.

Multiple Solutions

A reachable end-effector pose may correspond to more than one valid set of joint variables. Different manipulator configurations can therefore place the end-effector at the same position and orientation. Common examples include alternative elbow or shoulder configurations.

Where several solutions exist, the control system must select an appropriate configuration. This selection may depend on factors such as mechanical joint limits, proximity to the current configuration, collision avoidance, or the required direction of motion (Craig 2005).

Methods of Solution

Inverse kinematic solutions may be obtained using closed-form or numerical methods. Closed-form approaches provide a direct solution and can generally be divided into algebraic and geometric techniques. Algebraic methods manipulate the forward kinematic equations to isolate the unknown joint variables, while geometric methods use the physical geometry of the manipulator together with trigonometric relationships (Craig 2005).

Numerical methods instead determine a solution iteratively and are useful when a convenient closed-form solution cannot be obtained. Where the manipulator geometry permits it, however, a closed-form solution is advantageous because the required joint variables can be calculated directly without iterative convergence.

Inverse kinematics therefore requires consideration not only of how joint angles are calculated, but also whether the requested pose is physically achievable and whether more than one valid configuration exists.

2.7 Literature Review Summary

The literature shows that Industry 4.0 increasingly combines robotics, sensing, digital control, and automated decision-making to create manufacturing systems that are more flexible, consistent, and responsive. Robotic manipulators are well suited to inspection and sorting tasks, particularly when combined with machine vision and suitable end-effectors. Camera-based colour classification provides a practical method of identifying objects in a controlled environment, although its reliability can be affected by lighting, reflections, and object position. Gripper design and manipulator workspace are also critical because successful sorting requires both reliable object identification and dependable physical handling.

The review also highlights the importance of kinematic modelling in controlling serial robotic manipulators. Forward kinematics relates joint variables to the position and orientation of the end-effector, while inverse kinematics determines the joint configuration required to achieve a desired pose. Homogeneous transformations and the Denavit-Hartenberg convention provide systematic methods for describing manipulator geometry, while inverse kinematic solutions must also consider workspace, multiple possible configurations, and joint limitations. Low-cost platforms such as Arduino and rapid-prototyping manufacturing methods allow these principles to be demonstrated economically, although reduced stiffness, actuator performance, accuracy, and repeatability mean that such systems are best considered proof-of-concept platforms rather than substitutes for industrial robotic equipment.

3. System Requirements and Conceptual Design

This chapter defines the functional and engineering requirements of the robotic system and describes the process used to develop the final system architecture.

3.1 Requirements

The system requirements were defined to capture both the intended functional behaviour of the robotic sorting system and the principal engineering constraints required for reliable operation. The requirements were kept at a system level so that they could later be assessed during testing of the completed prototype.

Table 3.1: System requirements of the robotic sorting system.

ID	Requirement
R1	The system shall acquire image data from the vial inspection area, identify the colour present at each of the three predefined vial positions, and determine the corresponding sorting action.
R2	The manipulator shall provide sufficient workspace and positioning capability to reach all required pick-up, approach, and sorting locations.
R3	The control system shall calculate valid joint configurations for the required Cartesian target positions and prevent commanded movements outside the defined mechanical joint limits.
R4	The robotic arm shall automatically approach, grip, lift, transport, and release vials selected for sorting while leaving acceptable vials in their original positions.
R5	The manipulator and gripper shall support and retain one test vial throughout the complete sorting trajectory without excessive mechanical loading or loss of the object.
R6	The complete vision and sorting sequence shall operate automatically without continuous manual control, return to a defined safe position on completion, and be capable of being interrupted during development and testing if required.
R7	The prototype shall use low-cost, readily available components appropriate for proof-of-concept development.

3.2 Concept Development

The initial concept for the system was a fixed-base robotic manipulator capable of identifying and sorting coloured medical vials positioned within a predefined working area. The proposed system combined a four-degree-of-freedom serial manipulator, a separately actuated gripper, a fixed camera, and a computer-based control system. This configuration was selected to provide sufficient movement for the required pick-and-place task without introducing the additional mechanical and control complexity associated with a six-axis industrial manipulator.

The vial positions, sorting locations, camera, and robot base were fixed relative to one another. This allowed the vision system to determine the colour present at known inspection positions rather than requiring general-purpose object localisation. The resulting concept therefore concentrated the project effort on colour classification, robotic manipulation, and system integration.

3.3 Final System Architecture

The final system architecture combines the mechanical manipulator, embedded control electronics, machine-vision hardware, and high-level Python software into a single automated inspection and sorting system. The robotic arm is actuated by five servos: four control the rotational joints of the manipulator and one actuates the gripper. These servos are driven through a PCA9685 PWM controller, which is interfaced with an Arduino UNO R3. The Arduino provided the low-level hardware interface and a USB serial link to the host computer, while the Python software handled image processing, colour classification, sorting decisions, trajectory generation and inverse kinematics. The OV7670 camera supplied image data from the fixed inspection area.

3.4 Industry 4.0 Relevance

The completed system demonstrates several principles associated with Industry 4.0, particularly the integration of sensing, digital control, automated decision-making, and robotic manipulation within a single manufacturing process. Rather than following only a fixed sequence of movements, the robotic system first acquires information from the working environment and then adapts its subsequent action according to the detected vial colour.

The use of low-cost hardware also demonstrates how the underlying principles of intelligent automation can be implemented at proof-of-concept scale. While the system does not provide the accuracy, reliability, or networked IoT connectivity, of a commercial industrial installation, it provides a representative demonstration of how robotics, machine vision and software-based decision-making can be integrated within an automated manufacturing process.

3.5 Repurposing and Modular Design Principles

A further design consideration was the ability to modify and repurpose the system beyond the specific vial-sorting demonstration. The prototype was therefore developed using a

modular architecture in which the mechanical, electronic, and software subsystems could be modified independently rather than forming a single highly specialised system. The laser-cut arm structure, gripper, links, and supporting components could therefore be redesigned or replaced without requiring complete reconstruction of the manipulator.

The control architecture follows the same principle. The Arduino UNO R3 provides low-level hardware interfacing, the PCA9685 generates servo-control signals, and the host computer performs higher-level functions including machine vision, kinematic calculation, trajectory generation, and sorting logic. This separation allows individual elements to be upgraded or replaced while retaining the overall system structure; for example, the camera, gripper, or classification method could be changed without fundamentally redesigning the complete robotic platform.

The term modular architecture is used here in a project-specific sense to describe this separation and replaceability of functional subsystems. It is not intended to imply compliance with a formal open-architecture standard. These design choices allow the prototype to serve as a reusable experimental platform for other low-cost robotic inspection and handling tasks.

4. Mechanical Design

This section describes the mechanical design and manufacture of the robotic arm, including the manipulator geometry, revolute joints, gripper and the principal design iterations arising from physical prototyping.

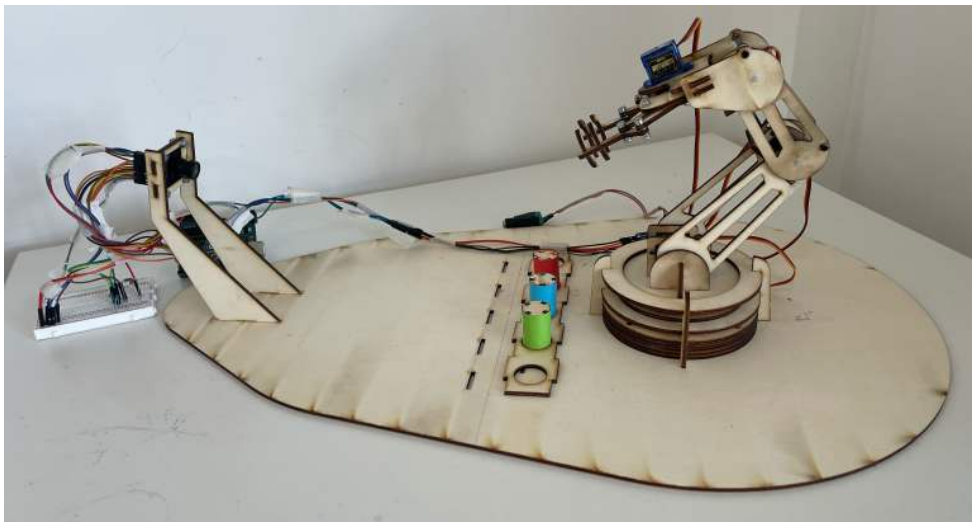


Figure 4.1: Full robot arm assembly.

4.1 Manipulator Configuration and Geometry

The manipulator was designed as a four-degree-of-freedom serial robotic arm using four revolute joints. The configuration consists of a rotating base followed by shoulder, elbow, and wrist joints, with a separately actuated gripper mounted at the end-effector. The base provides rotation about the global z-axis, while the shoulder, elbow, and wrist joints operate within the vertical plane established by the base rotation. This arrangement allows the end-effector to be positioned throughout the required three-dimensional workspace while maintaining a suitable orientation for vial handling.

A 4-DoF configuration was selected because it provides sufficient mobility for the intended pick-and-place task without introducing unnecessary mechanical or control complexity. The manipulator is required to reach predefined vial positions, approach them from above, grip them, and transport them to fixed sorting locations. Additional joints would increase dexterity, but would also increase mass, servo loading, cost, control complexity, and the number of possible inverse-kinematic solutions. The selected configuration therefore represents an appropriate compromise between mobility and simplicity for the proof-of-concept system.

The manipulator is mounted on a comparatively large base plate, which provides mechanical stability and also fixes the relative positions of the robot, camera, and vial inspection area. This fixed geometry is particularly important because the vision system classifies

colours within predefined image regions and the corresponding robotic pick-up positions are defined relative to the manipulator base.

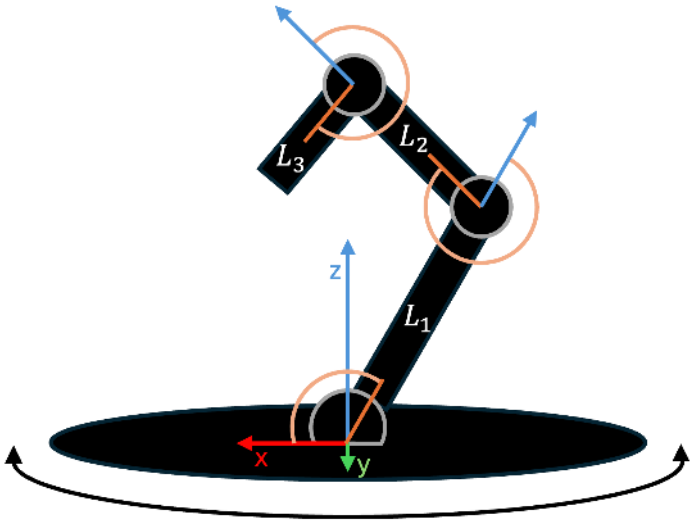


Figure 4.2: Manipulator configuration diagram.

The link dimensions were selected to provide sufficient reach for all required pick-up, approach, and sorting positions while keeping the structure compact and lightweight. The base raises the shoulder joint by 40 mm and provides approximately 180° of rotational travel. The first arm link, L_1, has a length of 100 mm and provides the majority of the manipulator's reach. The second link, L_2, is 60 mm and provides additional positioning flexibility between the elbow and wrist. The final effective link length, L_3, represents the distance from the wrist joint to the gripper end point, 105 mm and is therefore included explicitly in the kinematic model.

The end-effector uses a forward-facing pinch-gripper configuration. This allows the fingers to approach the cylindrical vial from the required direction while keeping the wrist and preceding links clear of the object. The gripper geometry was selected to provide a secure grasp within the limited workspace of the prototype and to remain compatible with the selected serial-link arrangement.

4.2 Joint Design

The four revolute joints were designed to provide the required range of motion while remaining mechanically simple and compatible with low-cost hobby servos. Different support and actuation arrangements were used according to the load and geometric requirements of each joint.

4.2.1 Base Joint

The base joint provides approximately 180° of rotation about the global z-axis. A circular rotating platform, approximately 90 mm in diameter, is supported on a ring of ball bearings to reduce friction and support the mass of the upper structure. Actuation is provided

by a servo mounted beneath the base plate, which directly controls the first joint angle, θ_1.

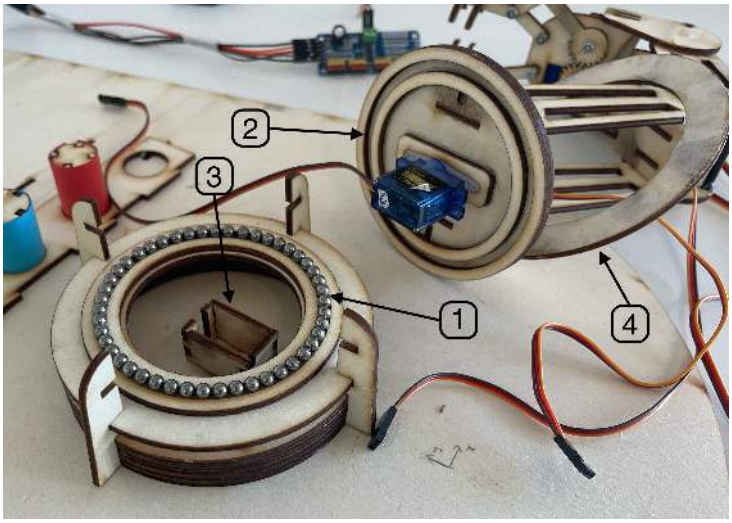


Figure 4.3: Deconstructed view of the base unit. (1) Ball bearings sitting in base tracks. (2) Upper bearing track. (3) Base servo mount. (4) Base clamping ring, secures the arm down onto the bearings.

4.2.2 Shoulder Joint

The shoulder joint provides 120° of usable rotation, from 30° to 150°, about the local y-axis. A servo mounted to the base directly drives one side of the first arm link through its horn, while a wooden dowel on the opposite side acts as a supporting pivot between the two parallel link members. This arrangement provides additional structural support and avoids relying solely on the servo output shaft to carry the joint load.

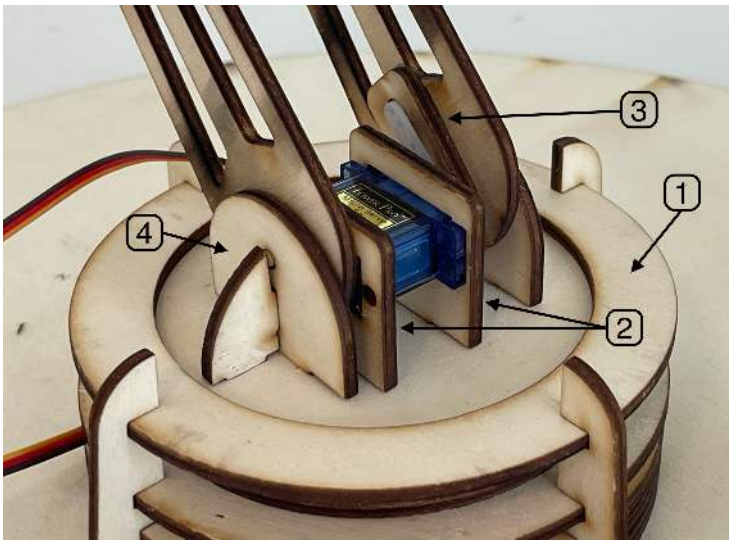


Figure 4.4: The shoulder joint. (1) Base clamp ring. (2) Shoulder servo mount. (3) Servo to arm actuation interface. (4) Shoulder structural support.

4.2.3 Elbow Joint

The elbow joint connects the first and second arm links and provides up to 90° of rotation about the local y-axis. A servo mounted at the end of the first arm link drives the second link through a custom linkage connected to both parallel members, allowing the actuator to be integrated within the arm structure while maintaining the required relative rotation between the links.

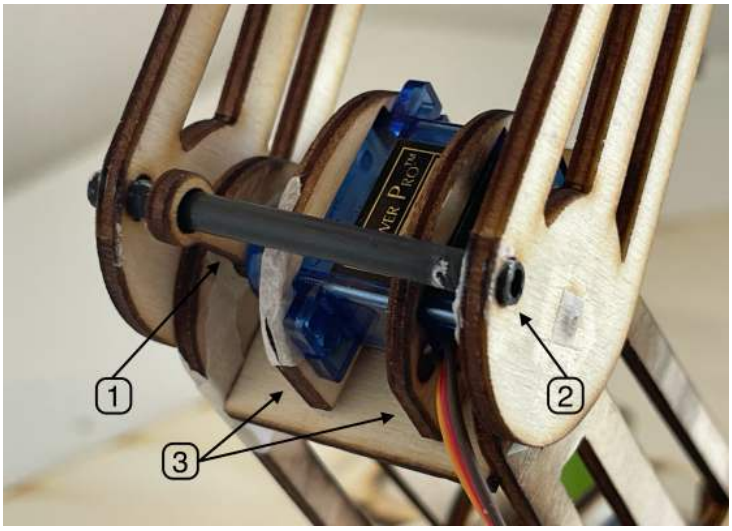


Figure 4.5: The elbow joint. (1) Custom servo actuation arm. (2) Carbon fibre actuation pin. (3) Elbow servo mounts.

4.2.4 Wrist Joint

The wrist joint provides a further 90° of rotation about the local y-axis between the second and third arm links. Its servo is mounted at the end of the second link and uses a similar custom linkage to drive both members of the third link. This joint provides the final orientational adjustment of the end-effector before the gripper.

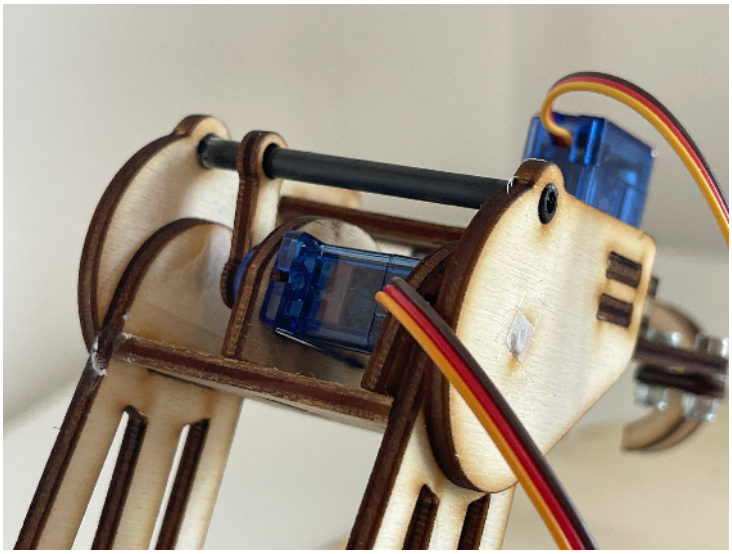


Figure 4.6: The wrist joint using fundamentally the same mechanism as the elbow.

4.3 Gripper Design

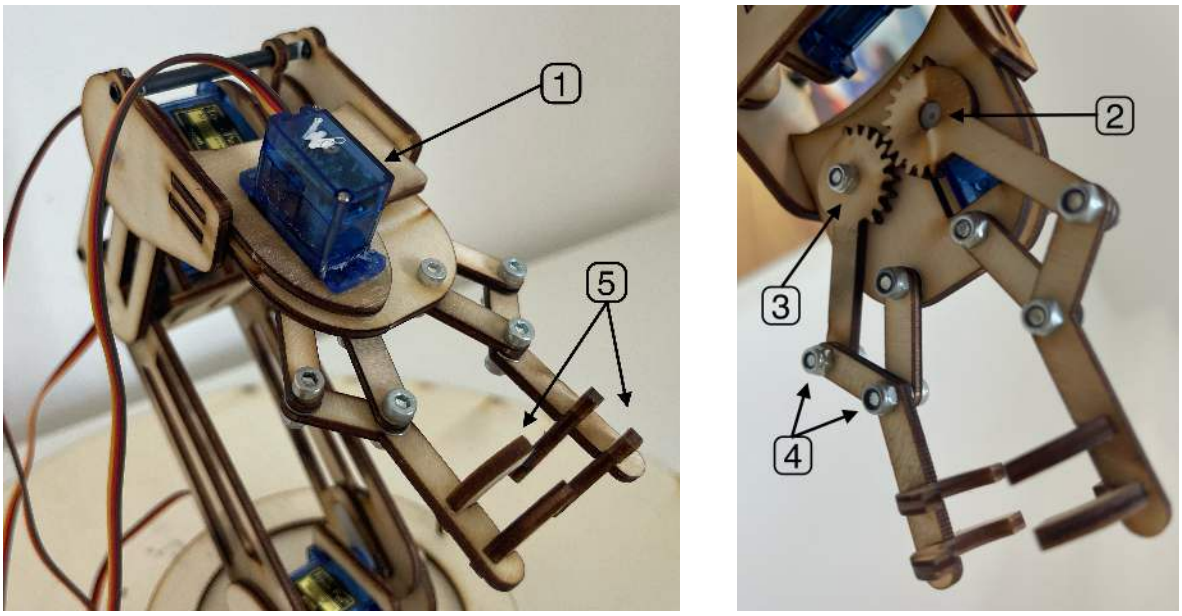


Figure 4.7: The gripper and its operating mechanism. (1) Gripper servo. (2) Servo actuated gripper linkage. (3) Second linkage actuated by gear. (4) Nyloc-nuts. (5) Curved finger ribs.

The gripper was designed specifically for the cylindrical medical vial used in the sorting task. Its geometry was therefore tailored to the vial dimensions rather than developed as a general-purpose end-effector. The inner finger surfaces incorporate shaped ribs that partially wrap around the vial, increasing contact area and reducing the likelihood of slipping during transport.

Actuation is provided by a servo mounted on the third arm link. The servo directly drives one finger, while the opposing finger is moved through a custom geared linkage so that both fingers close together using a forward pinching motion. This single-actuator arrangement provides coordinated motion while keeping the mechanism mechanically simple and low-cost. Nyloc-nuts are used to secure the linkage joints, reducing the likelihood of fastener loosening caused by repeated cyclic motion.

The forward pinching action allows the gripper to approach the vial from a controlled direction and close around its sides while maintaining the object between the two fingers. The resulting design is therefore closely matched to the handling requirements of the prototype sorting task rather than intended as a universal robotic gripper.

4.4 CAD Development

The mechanical components of the robotic arm were modelled in CREO Parametric to visualise the complete assembly and refine component dimensions, joint geometry, and mounting positions before manufacture. The parametric model also allowed design changes to be incorporated quickly and checked for fit and interference before physical parts were produced.

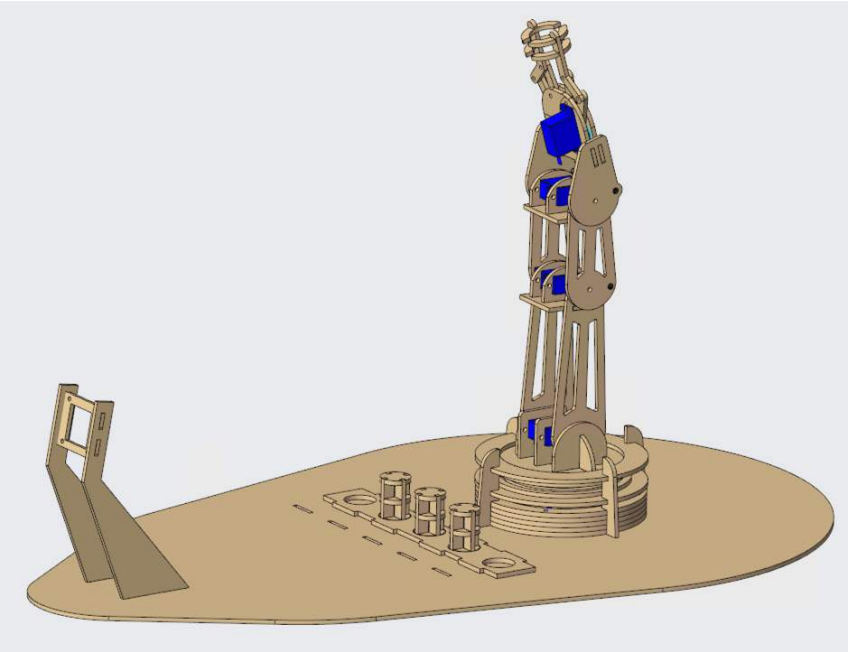


Figure 4.8: CREO Parametric CAD model of the complete robotic arm assembly.

Once the geometry had been finalised, the CAD models were used to generate the component profiles and technical drawings required for manufacture, including the files used for laser cutting. The CAD model therefore provided a direct link between design development and fabrication, helping to reduce dimensional errors and wasteful rework during assembly.

4.5 Laser-Cutter Manufacturing

Laser cutting was used as the primary manufacturing method for the structural components of the robotic arm, with most parts produced from 3 mm plywood. The process was well suited to the prototype because CAD-generated profiles could be manufactured quickly, consistently, and at low cost. It also supported rapid design iteration, as revised parts could be produced with minimal additional setup when changes to link geometry, joint features, or mounting positions were required.

Before final manufacture, tolerance testing was carried out for features such as slots, holes, and mating interfaces. Because the final fit can be affected by laser kerf and variation in material thickness, test features were produced and evaluated experimentally before the final dimensions were selected.

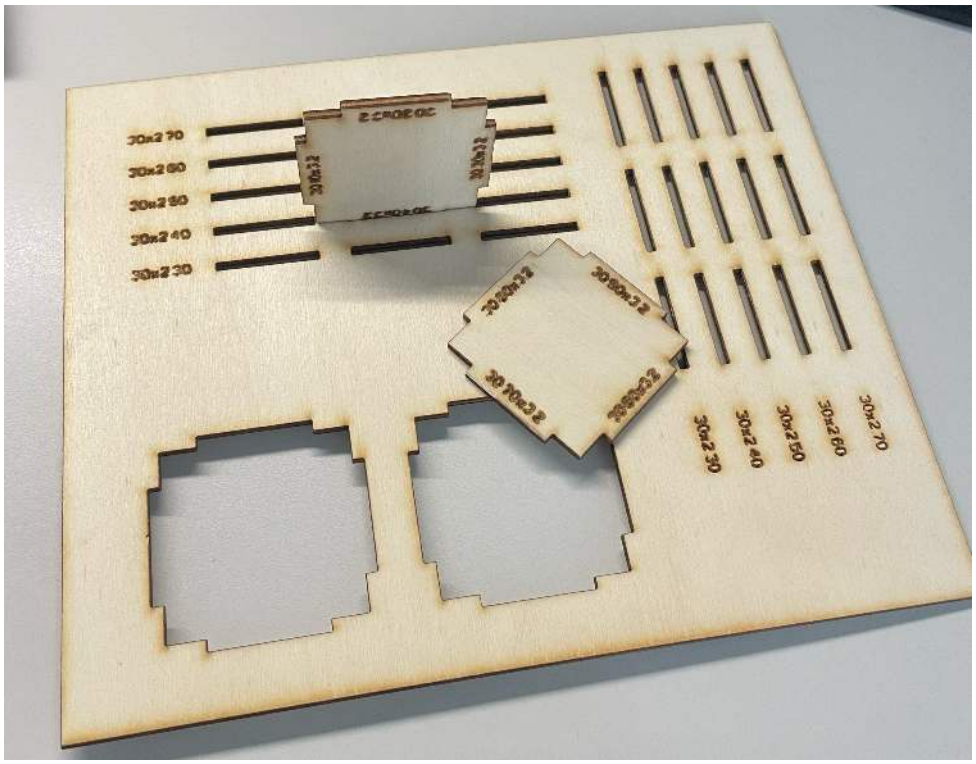


Figure 4.9: Custom-designed laser-cutter tolerance tester.

This process helped ensure that the components fitted securely while still allowing the required movement at the revolute joints, making laser cutting a practical rapid-prototyping method for the development of the manipulator.

4.6 Mechanical Design Iterations

The mechanical design was developed through a series of prototype iterations. Several issues only became apparent once the arm had been physically assembled and tested, particularly those relating to friction, component mass, joint fit, and ease of assembly. These observations were used to refine the design before the final prototype was completed.

The initial base design did not incorporate a bearing-supported rotating platform. As a result, friction between the rotating base components was sufficiently high that the base servo could not reliably overcome the resistance. The design was subsequently modified to include a ring of ball bearings beneath the rotating base disc. This reduced friction and allowed the base joint to rotate more smoothly and consistently.

The original gripper design also proved unsuitable because it contained a relatively large number of individual components. The resulting end-effector was heavier than anticipated and increased the moment applied to the shoulder joint. During testing, the shoulder servo was unable to reliably lift the complete arm with this additional mass. The gripper was therefore simplified to reduce its overall weight while retaining the required gripping function.

Several iterations were also required for the geared gripper mechanism. In the early versions, the gear teeth did not mesh consistently, resulting in excessive backlash. The gear

geometry and spacing was progressively adjusted until reliable meshing was achieved with reduced backlash while avoiding the mechanism binding and still allowing free movement of the fingers.

The base structure was also revised to improve manufacturability and assembly. The original design used vertical reinforcing ribs that fitted into slots in the base plate. Although this arrangement was suitable in the CAD model, the physical parts were difficult to align and assemble because of the close-fitting slot geometry. The base was therefore modified to simplify the assembly process and reduce the sensitivity of the structure to manufacturing tolerances.

A further limitation arose from the available servo torque. The full-size medical vials originally considered for the prototype increased both the required arm reach and the payload that had to be lifted. Since increasing the arm dimensions would also increase the joint moments, the demonstration objects were redesigned to reproduce only the section of the vial required for the gripping and sorting task. This reduced both the object mass and the required manipulator size while preserving the relevant gripping geometry.

These iterations demonstrate the importance of physical prototyping within the design process. Although the initial CAD models established the overall geometry, practical factors such as friction, actuator torque, backlash, manufacturing tolerance, and assembly accessibility could only be fully evaluated once the components had been manufactured and tested.

5. Control System

This chapter describes the hardware and software developed to control and simulate the robotic arm. It covers manipulator hardware components, kinematics, trajectory generation, and communication between the high-level software and embedded controller.

5.1 System Architecture

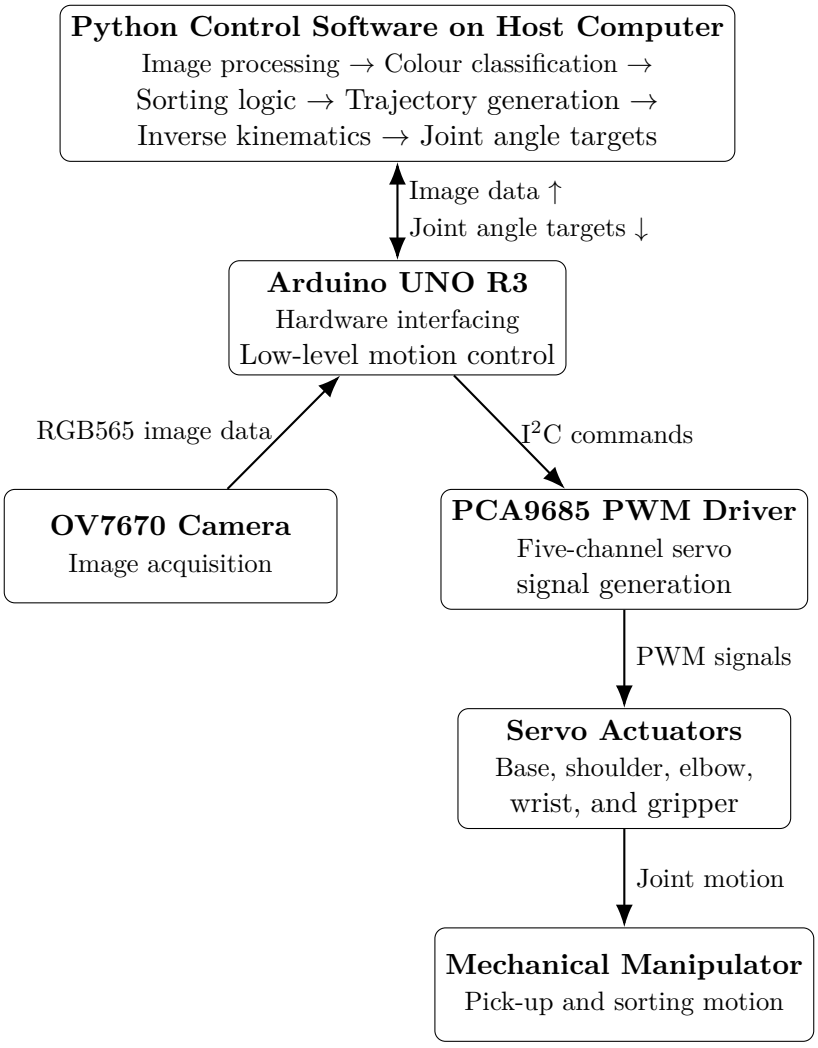


Figure 5.1: Functional architecture and control-data flow of the completed robotic inspection and sorting system.

Figure 5.1 illustrates the functional architecture of the completed system. High-level processing is performed by the Python program on the host computer, while the Arduino UNO R3 provides the interface between the software and the physical hardware. The OV7670 supplies image data for vial classification performed by the host computer, while

servo commands are transmitted through the PCA9685 PWM driver to the five actuators controlling the manipulator and gripper.

5.2 Control Hardware

5.2.1 Arduino UNO R3

The Arduino UNO R3 was used as the low-level embedded controller for the manipulator. The board is based on the ATmega328P microcontroller and operates at 16 MHz using 5 V logic. It provides USB serial communication with the host computer and an I^2C interface for communication with the PCA9685 servo driver.

The Arduino was not used for computationally intensive functions such as inverse kinematics, trajectory generation, or machine-vision processing, which were performed in Python on the host computer. Its role was instead to receive commands, interface with the peripheral hardware and coordinate low-level actuator motion. The UNO was suitable for this function because of its low cost, extensive documentation, and straightforward integration with the PCA9685 and other prototype hardware.

5.2.2 PCA9685 PWM Driver

A PCA9685 16-channel, 12-bit PWM driver was used to generate the repeated control pulses required by the five servos. The device communicates with the Arduino over I^2C and therefore allows multiple servo channels to be controlled using only the SDA and SCL communication lines.

Generating the servo timing on the PCA9685 reduces the timing burden on the Arduino and provides consistent multi-channel pulse generation. The five servos were assigned to separate PCA9685 channels as shown in Table 5.1.

Table 5.1: PCA9685 servo channel allocation.

Joint	PCA9685 Channel
Base	0
Shoulder	1
Elbow	2
Wrist	3
Gripper	4

5.2.3 Servo Actuation

Low-cost positional hobby servos were used to actuate the four revolute joints and the gripper. Servo position is commanded through the duration of a repeated control pulse, with the Arduino calculating the required calibrated position before passing the corresponding command to the PCA9685.

Individual calibration was required because the usable angular range and mechanical zero position differed between joints. Some servos were also mounted in reversed orientations,

requiring the commanded angle to be inverted in software. Hobby servos were appropriate for the proof-of-concept system because of their low cost and simple position control, although their limitations include backlash, limited torque, non-linearity, and the absence of externally accessible joint-position feedback.

5.2.4 Power Arrangement

The Arduino was powered from the host computer through USB, while the servos were powered from a separate external supply because their current demand exceeded that which could safely be supplied through the Arduino. The Arduino, PCA9685, and servo supply shared a common electrical ground to provide a common reference for the control signals. Separating the actuator and logic supplies also reduced the likelihood of voltage drops, resets and servo-induced electrical instability.

5.2.5 OV7670 Camera Interface

An OV7670 camera module was used to acquire the image data required for vial colour identification. The module operates at 3.3 V and provides 8-bit parallel pixel data together with synchronisation signals including PCLK, VSYNC, and HREF. Camera configuration is performed through the SIOD and SIOC control lines.

Compared with a simple colour sensor, the OV7670 requires a considerably larger number of electrical connections and tighter timing coordination because image data are transferred as a parallel pixel stream. The camera therefore forms the principal sensing interface within the hardware architecture, while the image processing and colour-classification methods are described separately in the machine-vision chapter.

5.2.6 Wiring

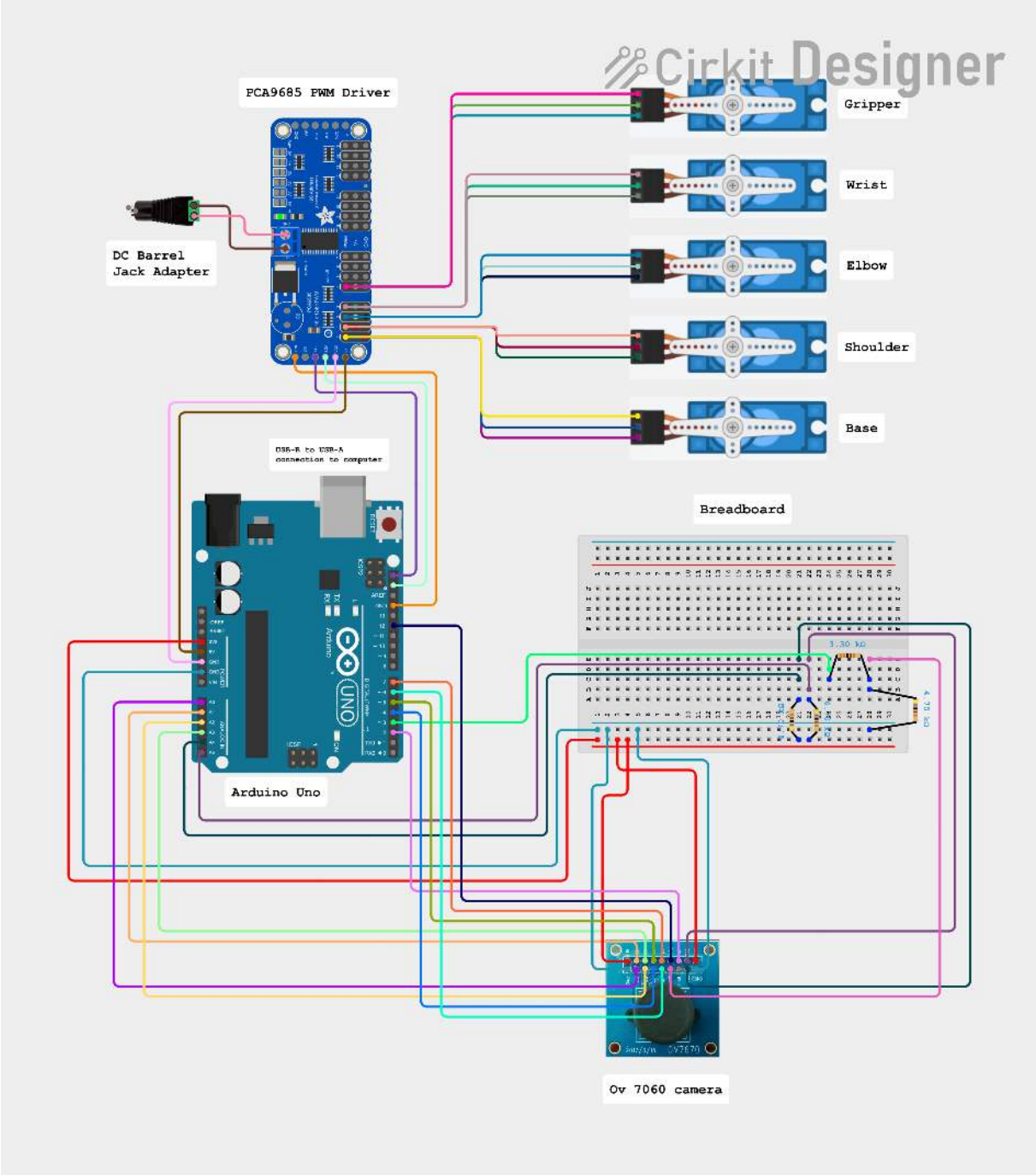


Figure 5.2: Wiring diagram of the robotic arm control system, produced in Cirkit Designer (Cirkit Designer 2026). For online view follow: https://app.cirkitdesigner.com/project/0900e36c-803b-4681-b52b-90e4940d26d2

5.3 Kinematics and Manipulator Control Software

This section describes the application of forward and inverse kinematics in the manipulator control software, along with the built in motion control and safety features designed to improve the operating capability of the arm.

All code relating to forward and inverse kinematics, joint-limit enforcement, trajectory generation, and three-dimensional arm visualisation was developed by the author. Generative AI tools were used to assist with the implementation of the Python to Arduino serial communication interface and with portions of the Arduino control software. The required system behaviour, communication structure, and integration of these components with the wider robotic control system were specified and validated by the author.

5.3.1 Coordinate Systems

A Cartesian coordinate system was defined relative to the base of the robotic arm to describe the position of the end-effector. The global z-axis extends vertically from the base, while the x- and y-axes define the horizontal working plane. End-effector positions are therefore specified using the Cartesian coordinates (x, y, z). Four joint angles, θ_1, θ_2, θ_3, and θ_4, are used to describe the configuration of the arm.

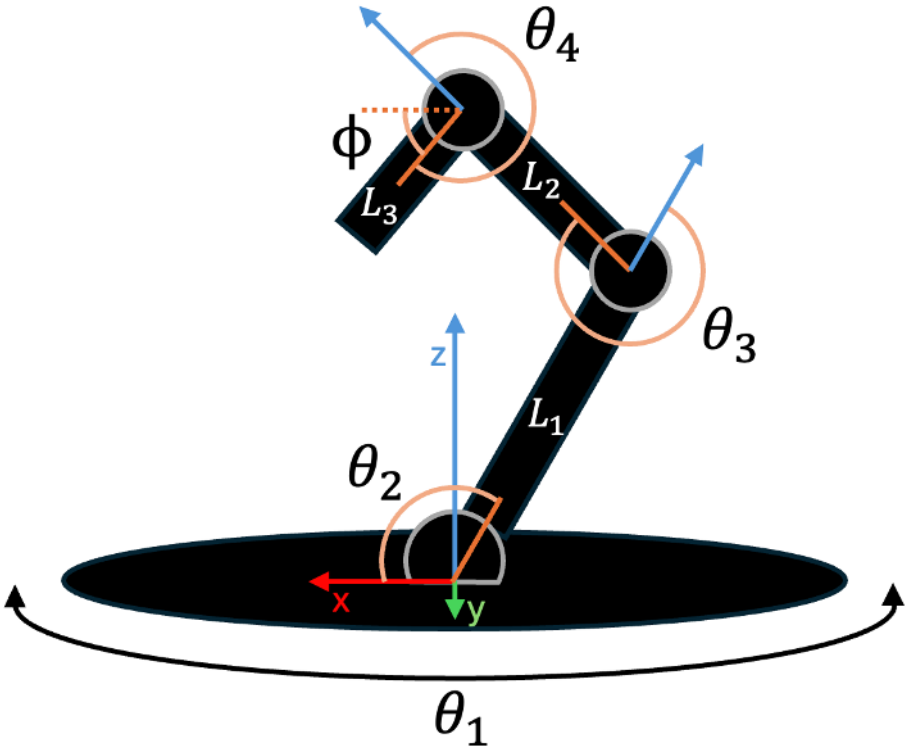


Figure 5.3: Diagram detailing how the code handles link angles relative to previous frame orientation.

5.3.2 Forward Kinematics

The theoretical forward kinematic relationship can be represented using successive homogeneous transformation matrices, as discussed previously. However, because the geometry of the developed manipulator is relatively simple, the transformation equations can be expanded into direct trigonometric expressions. The Python implementation therefore calculates the Cartesian coordinates of each joint directly rather than performing repeated 4×4 matrix multiplications.

```
143     def forward_kinematics(theta1, theta2, theta3, theta4):
144         # Convert angles from degrees to radians
145         t1 = np.radians(theta1)
146         t2 = np.radians(theta2)
147         t3 = np.radians(theta3)
148         t4 = np.radians(theta4)
149 
150         # Set base coordinates (origin)
151         x0 = 0
152         y0 = 0
153         z0 = 0
154         coord0 = np.array([x0, y0, z0])
155 
156         # Calculate the position of the shoulder (joint 1)
157         x1 = 0
158         y1 = 0
159         z1 = base_height
160         coord1 = np.array([x1, y1, z1])
161 
162         # Calculate the position of the elbow (joint 2)
163         x2 = x1 + L1 * np.cos(t1) * np.cos(t2)
164         y2 = y1 + L1 * np.sin(t1) * np.cos(t2)
165         z2 = z1 + L1 * np.sin(t2)
166         coord2 = np.array([x2, y2, z2])
167 
168         # Calculate the position of the wrist (joint 3)
169         x3 = x2 + L2 * np.cos(t1) * np.cos(t2 + t3)
170         y3 = y2 + L2 * np.sin(t1) * np.cos(t2 + t3)
171         z3 = z2 + L2 * np.sin(t2 + t3)
172         coord3 = np.array([x3, y3, z3])
173 
174         # Calculate the position of the gripper end (joint 4)
175         x4 = x3 + L3 * np.cos(t1) * np.cos(t2 + t3 + t4)
176         y4 = y3 + L3 * np.sin(t1) * np.cos(t2 + t3 + t4)
177         z4 = z3 + L3 * np.sin(t2 + t3 + t4)
178         coord4 = np.array([x4, y4, z4])
179 
180         return np.array([coord0, coord1, coord2, coord3, coord4])
```

Listing 5.1: Python implementation of the forward kinematics function.

Line No.	Explanation [Listing 6.1]
143 – 148	The function accepts the four joint angels in degrees and converts them into radians. Degrees are used elsewhere in the software as they are intuitive to relate to the physical servo positions, whereas NumPy trigonometric functions require angles in radians.
150 – 160	The fixed base frame is defined at the Cartesian origin $(0, 0, 0)$. The shoulder joint lies directly above the origin at height h, giving the coordinate $(0, 0, h)$. These two points establish the fixed reference frames from which the remaining arm geometry is calculated from.

162 – 166	The elbow coordinate is calculated relative to the shoulder position and the length of the first link, L_1. The term $L_1 \cos(\theta_2)$ gives the horizontal projection of the shoulder-to-elbow link. This radial component is resolved into the global $x-$ and $y-$directions using the base angle θ_1, while $L_1 \sin(\theta_2)$ gives the corresponding vertical displacement.
168 – 172	The wrist position is calculated relative to the previously determined elbow coordinate. Because the elbow angle is defined relative to the angle of the proceeding link, the absolute orientation of the second link within the arm plane is $\theta_2 + \theta_3$. The contribution of of L_2 to the wrist position is therefore calculated using the cumulative angle, rather than θ_3 alone.
174 – 178	The end-effector position is then calculated by adding the contribution of the final link L_3 to the wrist position. Its absolute orientation within the local arm plane is the sum of $\theta_2 + \theta_3 + \theta_4$.

It should be noted that the software implementation defines the planar joint angles relative to the horizontal axis. Consequently, $\theta_2 = 0°$ corresponds to the shoulder-to-elbow link being horizontal. This differs from the alternative vertical-zero convention used in the preceding theoretical discussion and must therefore be accounted for when relating the mathematical model to the physical servo calibration.

5.3.3 Inverse Kinematics

The inverse kinematics function converts a desired end-effector position (x, y, z, ϕ) into the four joint angles required to reach that position. The calculation exploits the geometry of the arm by first separating the base rotation from the remaining planar linkage. Once the base has been aligned with the target, the shoulder, elbow, and wrist can be solved as a two dimensional geometric problem. The implementation therefore uses direct trigonometric relationships rather than numerically inverting the forward kinematic model.

```
 97      def inverse_kinematics(x, y, z, phi):
 98          # Convert phi to radians
 99          phi = np.radians(phi)
100
101          # Calculate base rotation angle
102          theta1 = np.arctan2(y, x)
103
104          # Calculate the horizontal distance from the base to the
                target point
105          r4 = np.sqrt(x**2 + y**2)
106          # Calculate the vertical distance from the base to the target
                 point
107          h4 = z - base_height
108
109          # Calculate the position of the wrist (joint 3)
110          r3 = r4 - L3 * np.cos(phi)
111          # Calculate the vertical position of the wrist (joint 3)
112          h3 = h4 - L3 * np.sin(phi)
```

```
113
114         # Reachability check
115         d = np.sqrt(r3**2 + h3**2)
116         if d > (L1 + L2) or d < abs(L1 - L2):
117             raise ValueError("Target position is out of reach")
118
119         # Calculate elbow angle (theta3) using cosine rule
120         theta3 = np.pi + np.arccos((L1**2 + L2**2 - d**2) / (2 * L1 *
                L2))
121
122         # Calculate shoulder angle (theta2)
123         theta2 = np.arccos((L1**2 + d**2 - L2**2) / (2 * L1 * d)) +
               np.arctan2(h3, r3)
124
125         # Calculate wrist angle (theta4)
126         theta4 = phi - theta2 - theta3
127
128         # Convert to degrees
129         theta1 = np.degrees(theta1)
130         theta2 = np.degrees(theta2)
131         theta3 = np.degrees(theta3)
132         theta4 = np.degrees(theta4)
133
134         # Normalise angles to be within 0-360 degrees
135         theta1 = normalise_angle(theta1)
136         theta2 = normalise_angle(theta2)
137         theta3 = normalise_angle(theta3)
138         theta4 = normalise_angle(theta4)
139
140         return theta1, theta2, theta3, theta4
```

Listing 5.2: Python implementation of the inverse kinematics function.

Line No.	Explanation [Listing 6.2]
97 – 99	The function accepts the Cartesian coordinates and orientation of the end-effector required to perform the inverse kinematics calculation. The orientation, ϕ, is input in degrees to maintain intuitiveness, as the value must be manually coded for the specific movement function, and is converted to radians before use with NumPy's trigonometric functions.
101 – 102	The program determines base rotation independently from the remaining joints. The function `np.atan2` is used because it accounts for the signs of both Cartesian coordinates and therefore returns the correct target direction throughout the horizontal plane.
104 – 107	Once θ_1 has the arm pointed towards the target, the individual x and y coordinates can become one radial distance, r_4. The fixed base height is subtracted from z to express the vertical target position relative to the shoulder joint.

109 – 112	Since the position and orientation of the end of L_3 (the gripper), it is easy to find the position of the wrist joint. Solving this first will subsequently make calculating the shoulder and elbow geometry far simpler. It should be noted that the angle ϕ is measured from the horizontal, with a negative angle pointing the end-effector down.
114 – 117	This is a quick and simple safety check to ensure that the requested wrist position can physically be reached by the arm. If d exceeds the combined $L_1 + L_2$ length then the target is too far. Conversely, if $d < \|L_1 - L_2\|$, the target is within the minimum radius that can be produced by the two-link mechanism. In either case, execution of the program is halted.
119 – 120	The elbow angle is calculated using the cosine rule applied to the triangle formed by L_1, L_2, and d. The additional π term converts the internal triangle angle into the angular convention best suited for this program.
122 – 123	The shoulder angle is calculated from two components. The `arcos` term determines the angle between L_1 and the shoulder-to-wrist line, d, using the cosine rule, while `np.arctan(h3, r3)` determines the inclination of that line relative to the horizontal. The two components sum to the required shoulder rotation.
125 – 126	Once the shoulder and elbow angles have be determined, the wrist angle is calculated to comply with the specified end-effector orientation. Since the planar joint rotations accumulate through the serial chain, the wrist angle can be calculated from the angles of ϕ, θ_2, and θ_3.
128 – 132	The geometric calculations are performed in radians, while the remainder of the robot control system represents joint angles in degrees.
134 – 138	To ensure consistent angular representation the calculated angles are normalised within the range $0° - 360°$. This provides consistent angles for the subsequent joint-limit checks and stops equivalent angles, such as $-20°$ and $340°$, from breaking the program.

This implementation therefore avoids solving the complete homogeneous transformation matrix. Instead, the known geometry is used to reduce the three-dimensional inverse kinematics problem into a base-rotation and a two-dimensional planar linkage. This produces a closed-form solution requiring only trigonometric operations and avoids the computational expense and convergence uncertainty associated with iterative numerical inverse kinematics. The calculated angles remain theoretical kinematic values at this stage; physical servo orientation, calibration offsets, and mechanical joint limits are applied subsequently.

5.3.4 Joint Limits

The inverse kinematic solution may produce joint angles that are geometrically valid but cannot be achieved safely by the physical prototype. Each joint was therefore assigned a permissible angular range based on the available servo travel, mechanical construction, and the need to avoid self-collision or excessive loading.

Table 5.4: Joint limits.

Joint	$\theta_{\min} - \theta_{\max}$
Base	$0° - 180°$
Shoulder	$30° - 150°$
Elbow	$0° - 90°$
Wrist	$0° - 90°$
Gripper	$54° - 73°$

The base limit of 0°–180° reflects the usable range of the servo rather than the mechanical capability of the rotating structure. The remaining limits were selected experimentally to prevent self-collision, mechanical binding, and excessive loading of the joints.

Angles produced by the kinematic model are first normalised to the range 0°–360° and checked against the permitted joint limits. If any joint lies outside its allowable range, the target configuration is rejected before a command is sent to the Arduino. The relevant implementation is shown in Listing 5.3.

```
186     def normalise_angle(angle):
187         return angle % 360
188 
189 
190     def angle_within_limits(angle, lower_limit, upper_limit):
191         angle = normalise_angle(angle)
192 
193         # Full rotation case
194         if lower_limit == 0 and upper_limit == 360:
195             return True
196 
197         lower_limit = normalise_angle(lower_limit)
198         upper_limit = normalise_angle(upper_limit)
199 
200         # Normal case (30 to 150)
201         if lower_limit <= upper_limit:
202             return lower_limit <= angle <= upper_limit
203 
204         # Wrap-around case (300 to 60)
205         return angle >= lower_limit or angle <= upper_limit
```

Listing 5.3: Joint-angle validation and conversion for the physical servos.

The joint-limit checks therefore distinguish between geometric reachability and physical reachability. A Cartesian target may lie within the theoretical workspace of the manipulator but still be rejected if the required joint configuration exceeds one or more of the permitted mechanical ranges.

5.3.5 Trajectory Generation

Trajectory generation determines how the manipulator moves between two desired positions rather than only defining the start and end points. If the arm were commanded directly from one joint configuration to another, the end-effector path would depend on the individual joint motions and would not necessarily follow a predictable Cartesian route. For this project, interpolation was therefore used to generate a sequence of intermediate target coordinates between key positions.

For a start point

$$P_0 = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} \tag{5.1}$$

and an end point

$$P_1 = \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix} \tag{5.2}$$

a linear Cartesian trajectory can be written as

$$P_i = P_0 + s_i(P_1 - P_0) \tag{5.3}$$

where

$$s_i = \frac{i}{N}, \qquad i = 0, 1, \ldots, N \tag{5.4}$$

The interpolation parameter s is divided into a number of discrete increments. Each intermediate Cartesian pose is then passed to the inverse kinematics function, producing a sequence of joint-angle sets that approximate the required straight-line end-effector motion.

Increasing the number of intermediate points improves the approximation of a continuous Cartesian path, but also increases the number of inverse kinematic calculations and servo commands. Too few points can produce visibly segmented motion, while too many can introduce unnecessary communication and processing overhead.

Straight-line interpolation was particularly useful during approach and pick-up motions. Moving the gripper vertically or along a controlled linear path reduces the likelihood of deviation from the path and contact with the vial and provides more predictable object handling than unconstrained joint-space movement.

Linear interpolation defines the geometric path of the end-effector but does not by itself define the velocity profile. Motion speed is determined by the rate at which successive interpolated targets are issued to the actuator controller. In the prototype, this rate was selected experimentally to provide smooth movement while limiting vibration and excessive servo loading.

The trajectory-generation method therefore provides a simple Cartesian path-planning approach suited to the low-speed prototype. More advanced robotic systems may use higher-order polynomial, spline, or velocity-profile methods to control acceleration and

jerk explicitly, but linear interpolation was sufficient for the positioning and sorting tasks considered in this project.

5.3.6 Motion Smoothing

The servos used in the prototype are commanded by target position rather than direct velocity or acceleration commands. In the initial tests, a new target angle was written immediately to a servo, the actuator would attempt to move towards that position as quickly as its internal controller and available torque allowed. This produced abrupt movements, enough to damage some of the more fragile components.

To reduce this effect, the Arduino control program does not immediately apply a new target angle. Instead, the requested joint positions are stored as target values and each joint is moved towards its target in discrete one-degree increments. The rate at which these increments are applied determines the approximate commanded joint speed.

For the four manipulator joints, the maximum programmed speed is defined as

$$\omega_{\text{arm}} = 10 \ ^\circ/\text{s}. \tag{5.5}$$

The time interval between consecutive one-degree updates is therefore calculated from

$$\Delta t = \frac{1000}{\omega_{\text{arm}}}, \tag{5.6}$$

where Δt is expressed in milliseconds and ω_{arm} is expressed in degrees per second. For the selected speed of 10 °/s, this gives an update interval of

$$\Delta t = 100 \text{ ms}. \tag{5.7}$$

At each update, the current commanded angle of each joint is compared with its target. If the current angle is below the target it is increased by one degree; if it is above the target it is decreased by one degree. The process is repeated until the commanded and target positions are equal.

This may be represented as

$$\theta_{k+1} = \begin{cases} \theta_k + 1^\circ, & \theta_k < \theta_{\text{target}}, \\ \theta_k - 1^\circ, & \theta_k > \theta_{\text{target}}, \\ \theta_k, & \theta_k = \theta_{\text{target}}. \end{cases} \tag{5.8}$$

The motion update is implemented using the Arduino `millis()` function rather than a blocking delay. This allows the controller to continue checking serial input and updating other hardware while the arm is moving, rather than preventing execution of the remainder of the program during each movement.

All four arm joints are updated during the same control cycle. Consequently, joints requiring different angular displacements may reach their targets at different times, but each is limited to the same maximum angular rate. This provides a simple form of

coordinated motion while avoiding the abrupt position changes produced by direct servo commands.

The gripper is controlled separately from the arm joints and uses a lower programmed speed of

$$\omega_{\text{gripper}} = 5\ ^\circ/\text{s}. \tag{5.9}$$

This gives an update interval of approximately 200 ms per one-degree step. The lower speed was selected because the gripper mechanism has a relatively small operating range and is more sensitive to abrupt loading than the main arm joints.

The resulting control method provides a practical form of motion smoothing for the prototype without requiring explicit velocity or acceleration control from the servos. However, the method does not generate a true continuous acceleration profile. Each joint moves at approximately constant commanded angular velocity and therefore still experiences an abrupt change in velocity at the beginning and end of a movement. More advanced robotic manipulators commonly use trapezoidal or S-curve velocity profiles to control acceleration and jerk explicitly. Such methods could provide smoother dynamic behaviour but were not necessary for the low-speed proof-of-concept system developed in this project.

5.3.7 Servo Calibration

The joint angles produced by the kinematic model did not correspond directly to the raw servo commands because of differences in servo mounting, horn alignment, and mechanical assembly. Each joint was therefore calibrated experimentally and fixed angular offsets were introduced in the Arduino control program, as shown in Listing 5.4.

```
81    const int BASE_OFFSET     = -5;
82    const int SHOULDER_OFFSET = 14;
83    const int ELBOW_OFFSET    = 12;
84    const int WRIST_OFFSET    = 17;
85    const int GRIPPER_OFFSET  = 0;
```

Listing 5.4: Servo calibration offsets used in the Arduino control program.

The offsets were tuned with particular emphasis on the three predefined vial pick-up positions, as accurate alignment at these locations was critical for reliable gripping. Small adjustments were made until the physical arm consistently matched the required pick-up geometry.

The calibration should therefore be considered task-specific rather than a complete geometric calibration of the manipulator. Accuracy is prioritised within the regions used during the sorting sequence, while unused parts of the workspace may exhibit larger errors due to effects such as backlash, structural compliance, servo non-linearity, and manufacturing tolerances.

5.3.8 Safety and Emergency Stop

The prototype incorporates several software-based safety measures intended to reduce the risk of unintended or mechanically unsafe motion during development and testing. These measures build upon the joint-limit enforcement and controlled motion behaviour described previously.

Before a movement command is executed, the requested joint values are checked against the permitted angular ranges defined in Section 5.3.4. Any command containing an out-of-range value is rejected before the corresponding servo targets are updated. This prevents the controller from deliberately commanding configurations known to be outside the safe operating range of the mechanism.

The serial command parser also validates incoming data before movement is allowed. A valid motion command must contain the expected number of joint and gripper values in the correct format. Malformed, incomplete, or excessively long commands are rejected rather than being interpreted as actuator instructions. This reduces the possibility of unintended motion resulting from corrupted or incorrect serial data.

A further protective measure was incorporated into the pick-up manoeuvre to reduce excessive loading of the shoulder joint. During early testing, direct movement between some Cartesian positions caused the arm to pass through configurations in which the shoulder servo was required to support a large moment. To avoid this, specific intermediate Cartesian waypoints were introduced into the pick-up sequence so that the arm followed a safer path with reduced mechanical loading. These waypoints were selected experimentally and ensured that the manipulator approached and retracted from the vial without passing through configurations that risked overloading the shoulder servo.

An operator-controlled software stop was also incorporated into the host-side control program so that an active sorting sequence could be interrupted during testing. This was particularly important while calibrating the manipulator and evaluating new trajectories, where unexpected motion could otherwise continue until the programmed sequence had completed.

For the low-speed laboratory prototype, the combination of command validation, software joint-limit enforcement, conservative movement behaviour, and operator intervention was considered sufficient for controlled development and testing. However, further hardware-level safety measures, such as a means of electrically isolating the servo actuators from their power supply, would be required before the system could be considered suitable for industrial deployment.

5.3.9 Robot Simulation

A three-dimensional simulation of the manipulator was developed in Python using Matplotlib before the physical arm and electronic hardware were available. The simulation used the forward and inverse kinematic functions to visualise calculated joint positions and end-effector locations, providing an early means of checking angle conventions, link geometry, and joint orientation before these calculations were applied to the physical servos.

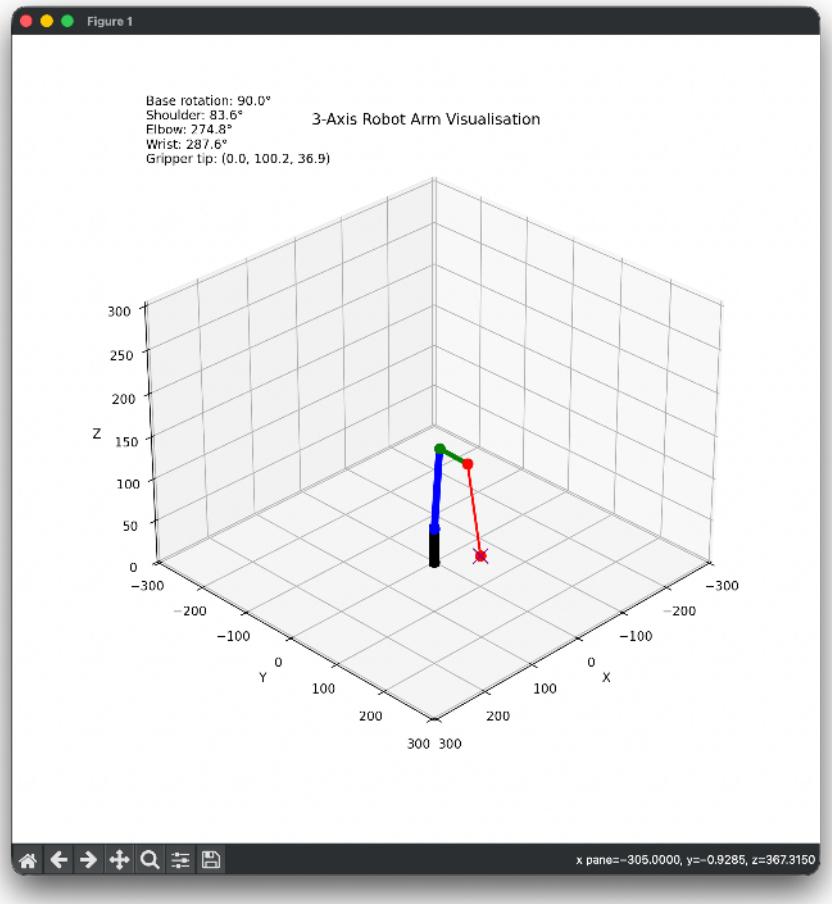


Figure 5.4: Three-dimensional Matplotlib visualisation of the robotic arm.

The simulator was also used to inspect multi-point trajectories by displaying the successive manipulator configurations generated between target positions. This was particularly useful when developing pick-up motions and checking that intermediate waypoints produced plausible arm configurations.

Following integration of the physical system, the simulation was retained as a diagnostic reference and updated using the same calculated joint positions sent to the real arm. Differences between simulated and physical motion could therefore indicate calibration, alignment, or actuator-related errors rather than faults in the underlying kinematic model. The simulation remained an ideal geometric model and did not include effects such as backlash, structural compliance, friction, or servo positioning error.

5.4 Machine Vision and Colour Classification

The vision system identifies the colour of the vial present at each of the three predefined positions and passes this information to the sorting logic. Because the camera and vial locations are fixed mechanically, the system does not require general-purpose object detection or coordinate estimation. Instead, fixed regions of interest within the image are analysed and classified as red, green, or blue.

To reduce the likelihood of an incorrect sorting decision, a result is only accepted when the expected combination of one red, one green, and one blue vial is detected consistently across multiple consecutive frames.

Generative AI tools were used to assist with the implementation of portions of the Python machine-vision code. However, the overall vision methodology was developed by the author, including the use of fixed regions of interest, median colour sampling, HSV-based classification, validation of the expected colour combination, and repeated-frame confirmation. The generated code was subsequently integrated, tested, and modified by the author as part of the completed system.

5.4.1 Image Acquisition

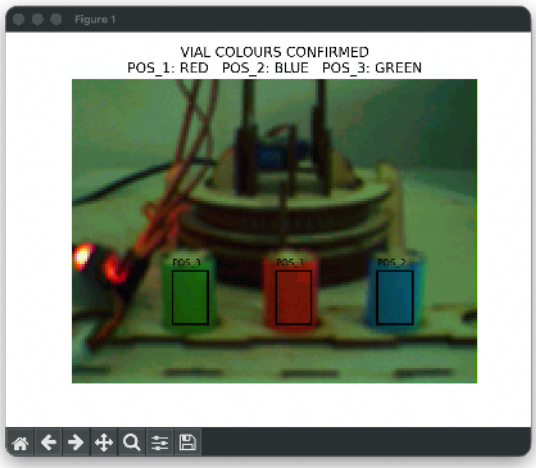
Image data is acquired from the OV7670 camera through the Arduino and transferred to the Python control program over the serial connection. Rather than streaming continuously, the host requests individual frames when required, reducing unnecessary data transfer while the manipulator is moving.

Each frame is transmitted in RGB565 format and reconstructed in Python as a three-channel image array for subsequent processing. A short startup delay is also included to allow the Arduino and camera to initialise and to prevent residual serial data from being interpreted as image information.

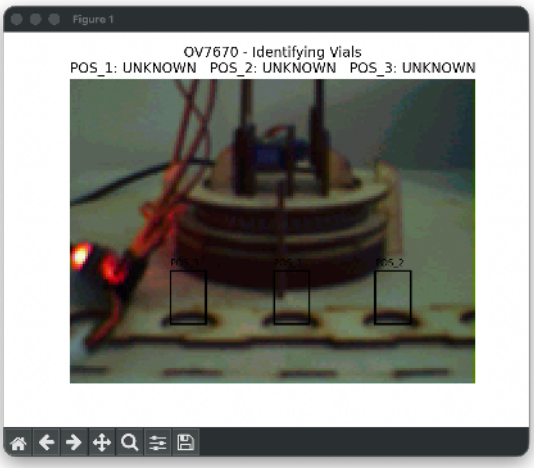
The resulting frames are then passed to the colour classification stage, where the predefined vial regions are analysed.

5.4.2 Region-of-Interest Colour Classification

Because the three vial locations are fixed relative to the camera, three predefined rectangular regions of interest (ROIs) are used rather than general-purpose object detection. Each ROI corresponds to one known vial position within the sorting area.



(a) Camera feed with the red, green, and blue vial colours successfully identified.



(b) Camera feed with no vials present, resulting in an `UNKNOWN` classification.

Figure 5.5: Machine-vision classification using the predefined regions of interest.

For each frame, the pixels within each ROI are analysed independently. The median red, green, and blue intensity values are calculated across the region rather than using a single pixel, reducing sensitivity to noise, reflections, and small local variations across the vial surface.

The median RGB values are then converted to HSV colour space and classified using predefined hue ranges. Low-saturation results are rejected as `UNKNOWN` to reduce false classification of poorly defined colours.

Table 5.5: Hue classification ranges.

Colour	Hue Range
Red	$hue < 30°$ or $hue > 330°$
Green	$90° < hue < 160°$
Blue	$170° < hue < 240°$

A classification is accepted only when the detected set contains one red, one green, and one blue vial and the same ordering is observed across multiple consecutive frames. This reduces the likelihood of transient lighting or camera noise producing an incorrect sorting decision.

5.4.3 Sorting Logic

Once the vial colours have been confirmed, the program converts the classification result into a predefined sorting sequence. Because both the vial locations and the sorting destinations are fixed, no image-to-robot coordinate transformation is required. Each vial position is instead associated with a known Cartesian pick-up coordinate and each sorting action with a predefined drop-off coordinate.

For each vial that must be moved, the appropriate pick-up and drop-off positions are selected and expanded into intermediate waypoints for approach, gripping, retraction, transport, release, and return to a safe transit position. Any vial assigned to remain in place is excluded from the movement sequence.

The vision system therefore determines what is present at each known location, while the motion-control system determines how the manipulator moves between the corresponding predefined coordinates.

5.4.4 Vision System Limitations

The vision system is dependent on a controlled operating environment, particularly consistent lighting. Low illumination can reduce colour saturation below the classification threshold, while strong back-lighting can distort the apparent vial colour and result in an `UNKNOWN` classification.

The method also assumes that one red, one green, and one blue vial are positioned within the three predefined ROIs. Duplicate or unexpected colours, missing vials, or significant displacement of either the camera or vial positions may therefore prevent a valid classification from being confirmed.

The system does not independently detect object presence, but instead classifies the colour content within each ROI. A more advanced implementation could combine colour classification with object or contour detection to identify missing, misplaced, or previously unseen objects and reduce dependence on fixed positioning.

5.5 Integration Challenges

Integrating the mechanical, electronic, vision, and software subsystems introduced a number of practical issues that were not apparent when each part was developed independently. The principal challenges involved camera reliability, communication timing, servo calibration, and ensuring that the theoretically generated trajectories were suitable for the physical manipulator.

5.5.1 Camera Hardware Fault

The most significant early vision-system problem was caused by the original OV7670 camera module. Despite continuity checks and verification of the relevant supply and control connections, the camera repeatedly produced an unusable image. After extensive troubleshooting, the module was replaced and the replacement camera operated correctly. Upon closer inspection it was determined that the voltage restrictor chip on the module was missing.

This issue demonstrated the difficulty of diagnosing integration faults where both hardware and software are being developed simultaneously. A failure within the camera module initially appeared indistinguishable from an error in wiring, camera configuration, or image-processing software.

5.5.2 Communication and Timing

A further challenge arose from the difference in processing speed between the Python control program and the embedded controller. The host computer could generate and transmit commands more rapidly than the Arduino and physical actuator system could reliably process them. This initially caused the simulated sequence and physical arm movement to become poorly synchronised.

The command rate and movement timing were therefore adjusted so that each stage of the control system had sufficient time to process and execute the requested motion. Camera initialisation also required dedicated startup handling to prevent residual serial data from being interpreted as part of a normal command or image frame.

5.5.3 Kinematic Model and Physical Servo Calibration

The forward and inverse kinematic models describe an idealised manipulator, whereas the physical prototype contains mounting offsets, reversed servo orientations and manufacturing tolerances. Consequently, mathematically correct joint angles did not initially correspond directly to the required physical positions.

Individual servo offsets and direction corrections were introduced experimentally to align the physical manipulator with the software model. The Python simulation was partic-

ularly useful during this process because it provided a reference for the expected arm configuration. Differences between the simulated and physical motion could therefore be used to identify calibration or mechanical issues rather than errors in the underlying kinematic calculations.

5.5.4 Trajectory and Actuator Loading

Some Cartesian movements that were geometrically valid still produced mechanically unfavourable arm configurations. In particular, certain direct movements placed excessive loading on the shoulder servo. Specific intermediate Cartesian waypoints were therefore incorporated into the pick-up sequence to prevent the arm from passing through these configurations.

This highlighted an important limitation of purely geometric path planning: a reachable trajectory is not necessarily mechanically suitable for the real system. The final motion sequence therefore had to account for both kinematic reachability and practical actuator loading.

5.5.5 Gripper and Motion Sequencing

Integrating the gripper with the complete sorting sequence also required adjustment of the timing between arm movement and gripper actuation. During development, the gripper could reach an incorrect state at the pick-up point if its opening and closing commands were not correctly synchronised with the arm trajectory.

The final sequence therefore explicitly defines the required gripper state at each waypoint, including approach with the gripper open, closure at the pick-up position, transport while closed, and release at the sorting location.

6. Testing and Results

This chapter presents the experimental procedures used to evaluate the performance of the completed system and reports the resulting measurements.

6.1 Testing Methodology

Testing was carried out on the completed robotic sorting system to evaluate the accuracy, repeatability, and reliability of the mechanical, control, and vision subsystems. The tests were designed to assess both individual subsystem performance and the ability of the complete system to perform the intended automated sorting task.

Practical, repeated trials were used rather than single measurements so that both accuracy and consistency could be assessed. Joint and Cartesian positions were commanded through the same software used during normal operation, while physical measurements were taken manually using appropriate measuring equipment. Vision and sorting performance were evaluated using repeated classifications and complete automated sorting cycles.

6.2 Joint Position Testing

Joint-position accuracy was assessed by commanding each joint to three angles distributed across its permitted range of motion. The selected values were intended to sample the lower, middle, and upper regions of each joint range rather than only testing a single operating position.

The resulting physical joint angles were measured manually using a protractor and compared with the commanded values. Because the measurements were read visually, the method provides only an approximate assessment of angular accuracy; however, this level of measurement was considered sufficient for evaluating the performance of the proof-of-concept prototype.

Table 6.1: Joint position accuracy results.

Joint	Commanded Angle (°)	Measured Angle (°)	Absolute Error (°)
Base	30	31	1
Base	90	90	0
Base	150	150	0
Shoulder	45	43	2
Shoulder	90	91	-1
Shoulder	135	137	-2
Elbow	15	14	1
Elbow	45	45	0
Elbow	75	75	0
Wrist	15	15	0
Wrist	45	45	0
Wrist	75	75	0

The joint-position tests produced a mean absolute error of 0.58° and a maximum absolute error of 2°. The largest deviations occurred at the shoulder joint, while the elbow and wrist reproduced most of the commanded test angles without measurable error. These results indicate that the calibrated joint commands provide sufficient angular accuracy for the intended proof-of-concept sorting task.

6.3 End-to-End Automated Sorting Test

The completed system was evaluated using repeated end-to-end sorting trials to assess the combined performance of the machine-vision, manipulator-control, and gripper systems. All six possible arrangements of one red, one green, and one blue vial were tested three times, giving a total of 18 trials.

Each trial was assessed at the main stages of the automated sequence: vision classification, red-vial pickup and placement, blue-vial pickup and placement, and return to the safe position. Each stage was recorded as a pass (P) when it was completed successfully without operator intervention, and as a fail (F) otherwise.

An overall pass was recorded only when every required stage of the sorting sequence was completed successfully. The green vial was intended to remain in its original position and was therefore considered successful when it remained undisturbed throughout the trial.

Table 6.2: End-to-end automated sorting test results.

Trial No.	Vial Order	Vision Success	Pickup Red	Place Red	Pickup Blue	Place Blue	Return	Overall Result
1	RGB	P	P	P	P	P	P	P
2	RGB	P	P	P	P	P	P	P
3	RGB	P	P	P	P	P	P	P
4	RBG	P	P	P	P	P	P	P
5	RBG	P	**F**	**F**	P	P	P	**F**
6	RBG	P	P	P	P	P	P	P
7	GRB	P	P	P	P	P	P	P
8	GRB	P	P	P	P	P	P	P
9	GRB	P	P	P	P	P	P	P
10	GBR	P	P	P	P	P	P	P
11	GBR	P	P	P	P	P	P	P
12	GBR	P	P	P	P	P	P	P
13	BRG	P	**F**	**F**	**F**	**F**	P	**F**
14	BRG	P	P	P	P	P	P	P
15	BRG	P	P	P	P	P	P	P
16	BGR	P	P	P	P	P	P	P
17	BGR	P	P	P	P	P	P	P
18	BGR	P	P	P	P	P	P	P

P = Pass, F = Fail.

The end-to-end testing produced an overall sorting success rate of 88.9%, with 16 of the 18 trials completed successfully without operator intervention. Vision classification and return-to-safe-position stages were successful in all 18 trials. Red-vial pickup and placement were each successful in 16 of 18 trials, while blue-vial pickup and placement were each successful in 17 of 18 trials. The two unsuccessful trials therefore resulted from failures during the physical manipulation stages rather than from the vision system or return sequence. The failures occurred from a small misalignment that caused a collision between the gripper and the vial during the pickup stage.

7. Discussion

This chapter critically evaluates the performance of the completed robotic system against the original project objectives and considers the extent to which the mechanical, control, and machine-vision subsystems met the set objectives and requirements. The experimental results are also considered in relation to the limitations of the low-cost hardware and methods identified in the literature review.

7.1 Achievement of Project Objectives

The overall aim of the project was to design, manufacture, and integrate a low-cost robotic system capable of automatically identifying and sorting coloured demonstration vials. This aim was substantially achieved. The completed prototype combined a four-degree-of-freedom manipulator, separately actuated gripper, machine-vision system, inverse-kinematic control, and automated sorting sequence within a single functioning system.

The first objective was to manufacture a manipulator capable of reaching the predefined pick-up and sorting positions. This was achieved, as successful sorting was demonstrated from all six possible vial arrangements during the end-to-end testing. Although two individual trials failed during vial pick-up, the same locations were successfully reached during other repetitions, indicating that the positions were within the practical workspace of the manipulator and that the failures arose from positioning variation, likely caused by the use of low-cost servos, rather than an unreachable target.

The second objective concerned integration of the kinematic, control, and machine-vision systems, was also achieved. The vision system successfully identified the vial arrangement in all 18 end-to-end trials, after which the Python control software generated the required sorting sequence and joint positions without continuous operator control. The completed system therefore demonstrated the intended sequence from visual inspection through automated decision-making to physical manipulation.

The third objective was to evaluate the prototype experimentally. Joint-position testing produced a mean absolute angular error of 0.58° and a maximum error of 2°, while the complete sorting system successfully completed 16 of 18 trials, corresponding to an overall success rate of 88.9%. The testing also identified physical positioning accuracy during vial pick-up as the principal remaining source of failure.

7.2 Mechanical Performance

The mechanical structure was ultimately capable of supporting the complete pick-and-place operation, but its development demonstrated several limitations associated with low-cost robotic prototyping. Early designs experienced excessive base friction, high gripper mass, gear backlash, and assembly difficulties. These problems required iterative redesign, including the addition of ball bearings to the base, reduction of gripper mass, and modification of the geared finger mechanism. This supports the literature finding

that rapid-prototyping methods are particularly valuable where components must be repeatedly modified during development.

Actuator loading was one of the most significant constraints on the final design. Increasing the mass or reach of the end-effector substantially increased the moment carried by the shoulder servo, and the original gripper and full-size vial geometry could not be operated without exceeding the practical capability of the actuator. The final design therefore represents a compromise between manipulator reach, structural mass, payload, and available servo torque. This is consistent with the limitations identified for low-cost robotic systems, where actuator torque, structural stiffness, backlash, and payload capacity are typically lower than those of industrial manipulators (Salazar et al. 2024).

The custom gripper proved suitable for the cylindrical demonstration vial during most trials. Its shaped finger geometry provided a secure grasp once correctly positioned, and no failures were recorded as a result of the vial slipping during transport. However, the two unsuccessful end-to-end trials resulted from small positioning errors that caused the gripper to collide with the vial during approach. This demonstrates that an application-specific gripper can improve object retention but cannot compensate fully for errors in manipulator positioning. The result is consistent with the importance of matching gripper geometry and contact conditions to the intended payload identified in the literature (Hernandez et al. 2023).

7.3 Control System Performance

The control architecture successfully converted predefined Cartesian target positions into physical manipulator motion. The closed-form inverse-kinematic solution was appropriate for the relatively simple four-DoF geometry and avoided the additional computational requirements of an iterative numerical solution. The forward-kinematic simulation also provided a useful development tool for checking the expected manipulator configuration before and during physical integration.

The joint-position results indicate that the calibrated servo commands reproduced the requested joint angles reasonably closely for a low-cost prototype. The mean absolute error was 0.58°, with a maximum measured error of 2°. The largest deviations occurred at the shoulder, while most elbow and wrist measurements showed no observable error. These measurements should nevertheless be interpreted cautiously because the angles were measured manually using a protractor, meaning that the resolution of the measurement method was limited.

The difference between the ideal kinematic model and the physical manipulator was more significant than the mathematical solution alone would suggest. Servo mounting offsets, reversed actuator orientations, backlash, structural compliance, and manufacturing tolerances required experimental calibration before calculated joint positions produced satisfactory physical alignment. Calibration was deliberately concentrated around the three vial pick-up locations, meaning that the reported performance should not be interpreted as uniform accuracy throughout the complete workspace.

Trajectory generation also required consideration of physical actuator loading. Some geometrically valid movements placed an unacceptable load on the shoulder servo, requiring intermediate Cartesian waypoints to be introduced. This illustrates an important distinc-

tion between kinematic reachability and practical robotic operation: a mathematically valid configuration is not necessarily mechanically desirable. A more advanced control system would incorporate dynamic loading, acceleration, and actuator capability directly into its trajectory planning rather than relying on experimentally selected waypoints.

7.4 Vision System Performance

The machine-vision subsystem performed reliably under the controlled conditions used for testing. Correct colour classification was achieved in all 18 end-to-end trials, indicating that the combination of fixed regions of interest, median RGB sampling, HSV classification, and repeated-frame confirmation was effective for the defined task.

This result supports the decision to avoid general-purpose object detection. Because the camera and vial positions were mechanically fixed, the system only needed to determine which colour was present at each known position. The simplified method therefore provided the information required by the sorting process with relatively low computational complexity, consistent with the use of colour-based classification for constrained sorting applications discussed in the literature (Shen and Hassan 2014).

However, the 100% classification rate observed during the trials should not be interpreted as evidence of equivalent performance under unrestricted operating conditions. The system was tested with the expected red, green, and blue vials at fixed locations and under a controlled imaging arrangement. The method remains sensitive to illumination, camera alignment, and vial position, as anticipated from the literature on camera-based inspection. It also assumes that exactly one vial of each expected colour is present and does not independently determine whether an object exists within an ROI. The strong experimental result therefore demonstrates reliability for the defined prototype task rather than general-purpose visual inspection capability.

7.5 Industry 4.0 Application

The completed prototype demonstrates the fundamental information flow associated with an Industry 4.0 robotic inspection process. Visual information is acquired from the physical environment, processed digitally to classify the objects, used to determine the required sorting action, and subsequently converted into physical robotic motion. The system therefore integrates sensing, automated decision-making, software control, and robotic manipulation rather than operating only as a pre-programmed sequence.

The architecture also demonstrates the benefit of separating higher-level processing from low-level hardware control. Python performs machine vision, sorting decisions, trajectory generation, and inverse kinematics, while the Arduino and PCA9685 provide the interface to the physical actuators. This modular arrangement allows individual elements of the prototype to be changed without requiring complete redevelopment of the system.

Nevertheless, the prototype represents Industry 4.0 principles only at proof-of-concept scale. It does not incorporate industrial communication protocols, networked data exchange, production-system integration, or continuous monitoring. Similarly, the use of predefined static vial positions avoids the localisation and real-time synchronisation requirements associated with conveyor-based industrial sorting. The system should there-

fore be considered as a demonstration of the underlying sensing, decision, and actuation process rather than an industrially deployable Industry 4.0 system.

7.6 System Limitations

The principal limitation of the prototype is its dependence on a highly constrained operating environment. Vial positions, camera orientation, robot position, and sorting destinations are all predefined. This simplified the vision and control problems sufficiently to allow a complete low-cost system to be demonstrated, but also restricts the ability of the robot to respond to objects placed at arbitrary locations.

The use of low-cost servos also limits positioning accuracy and repeatability. The actuators do not provide externally accessible joint-position feedback to the control software, meaning that commanded position is assumed to correspond to physical position. Errors arising from backlash, compliance, loading, or servo non-linearity therefore cannot be detected and corrected automatically. The two failed sorting trials demonstrate the practical consequence of this limitation: small positioning deviations were sufficient to cause contact between the gripper and vial during the critical pick-up operation.

The experimental evaluation itself also has limitations. Joint angles were measured manually using a protractor, and only three positions per joint were assessed. The end-to-end evaluation consisted of 18 trials, providing useful evidence of proof-of-concept reliability but not enough data to establish industrial levels of repeatability or statistical confidence. Testing was also performed using the same controlled arrangement for which the system had been calibrated.

These limitations do not prevent the prototype from meeting its intended purpose, but they define the boundary of the conclusions that can be drawn from the results. The system demonstrates successful integration of low-cost robotic manipulation, kinematic control, machine vision, and automated sorting, while further development would be required to achieve the positional accuracy, environmental robustness, feedback control, and flexibility expected of an industrial robotic system.

8. Conclusions and Future Work

This chapter summarises the principal outcomes of the project and identifies areas in which the robotic system could be developed further.

8.1 Conclusions

This project successfully designed, manufactured, and integrated a low-cost robotic inspection and sorting system as a proof-of-concept Industry 4.0 application. The completed system combined a four-degree-of-freedom manipulator, separately actuated gripper, machine-vision system, inverse-kinematic control, trajectory generation, and automated sorting logic within a single working prototype.

The manipulator was able to reach the required vial pick-up and sorting positions, while the machine-vision system correctly identified the vial arrangement in all 18 end-to-end trials. Joint-position testing produced a mean absolute error of 0.58° and a maximum error of 2°. The complete automated sequence was successfully completed in 16 of 18 trials, corresponding to an overall success rate of 88.9%. The unsuccessful trials resulted from small positioning errors during vial pick-up rather than failures of the vision system or sorting logic. These results demonstrate that the developed system was capable of performing the intended inspection and sorting task with a high level of reliability for a low-cost proof-of-concept prototype.

The project also demonstrated the importance of integrating theoretical kinematic modelling with practical mechanical and actuator considerations. Although the forward and inverse kinematic models provided valid manipulator configurations, servo calibration, mechanical tolerances, actuator loading, and experimentally selected waypoints were required to achieve reliable physical operation. Iterative development of the base, gripper, and control system was therefore essential to the final performance of the prototype.

Overall, the original project aim and objectives were achieved. The completed system demonstrates the integration of sensing, automated decision-making, digital control, and robotic manipulation in a form representative of the fundamental principles of Industry 4.0. However, the system remains a controlled laboratory prototype and does not provide the accuracy, robustness, flexibility, or connectivity expected of an industrial robotic installation.

8.2 Future Work

The most direct improvement would be to increase the positional accuracy and repeatability of the manipulator. Higher-quality actuators with reduced backlash and greater torque would improve joint performance, while the addition of independent joint-position sensing would allow closed-loop feedback to verify that the physical joint position matches the commanded position. Improved structural stiffness and more precise mechanical manufacture would further reduce deflection and alignment errors.

The vision system could also be developed beyond the current fixed-ROI approach. A higher-quality camera and improved lighting control would increase robustness, while object or contour detection could allow the system to identify vial position as well as colour. More advanced image-recognition or machine-learning methods could then enable the system to classify a wider range of objects and operate without requiring predefined inspection positions.

A further development would be the integration of a conveyor system. This would require the robot to detect and track moving objects and synchronise its motion with their position, providing a more representative industrial sorting application. The current placement sequence could also be improved by introducing more accurate drop-off control and feedback to confirm that each vial has been released successfully.

At system level, the prototype could be extended through industrial communication protocols and network connectivity so that operating data, classifications, and system status could be exchanged with other manufacturing equipment. A digital-twin model could also be developed to provide a continuously updated virtual representation of the physical manipulator for monitoring, simulation, and trajectory validation.

These developments would progressively reduce the dependence on fixed geometry and manual calibration, allowing the prototype to evolve from a constrained demonstration towards a more flexible and representative Industry 4.0 robotic inspection and handling system.

References

Abbood, Wisam T., Oday I. Abdullah, and Enas A. Khalid (2019). "A real-time automated sorting of robotic vision system based on the interactive design approach". In: *International Journal on Interactive Design and Manufacturing (IJIDeM)* 14.1, pp. 201–209. DOI: 10.1007/s12008-019-00628-w. (Visited on 06/29/2026).

Ajiga, Daniel et al. (2024). "The Role of Software Automation in Improving Industrial Operations and Efficiency". In: *International Journal of Engineering Research Updates* 7.1, pp. 022–035. DOI: 10.53430/ijeru.2024.7.1.0031. (Visited on 06/29/2026).

Borkar, Shweta et al. (2023). *A Review of Pick and Place Robots for the Pharmaceutical Industry*. DOI: 10.1109/ICETET-SIP58143.2023.10151652. (Visited on 06/29/2026).

Cirkit Designer (2026). *Cirkit Designer*. URL: https://app.cirkitdesigner.com/ (visited on 08/25/2026).

Craig, John J. (2005). *Introduction to Robotics: Mechanics and Control*. 3rd ed. Upper Saddle River, New Jersey: Pearson Prentice Hall.

De, Anurag et al. (2025). "WareRob: A Warehouse Management Robot". In: *2025 5th International Conference on Artificial Intelligence and Signal Processing (AISP)*. IEEE, pp. 1–5. DOI: 10.1109/aisp68263.2025.11396312. (Visited on 06/29/2026).

Hernandez, Jaime et al. (2023). "Current Designs of Robotic Arm Grippers: A Comprehensive Systematic Review". In: *Robotics* 12.1, p. 5. DOI: 10.3390/robotics12010005. (Visited on 06/29/2026).

Kapoor, Rohan (2023). *The Benefits of Using Arduino in Robotics and Automation*. URL: https://kunkune.co.uk/blog/the-benefits-of-using-arduino-in-robotics-and-automation/ (visited on 06/29/2026).

Koh, Lenny, Guido Orzes, and Fu (Jeff) Jia (2019). "The fourth industrial revolution (Industry 4.0): technologies disruption on operations and supply chain management". In: *International Journal of Operations & Production Management* 39.6/7/8, pp. 817–828. DOI: 10.1108/ijopm-08-2019-788. (Visited on 06/29/2026).

Magalhaes, Paulo and Nuno Ferreira (2022). "Inspection Application in an Industrial Environment with Collaborative Robots". In: *Automation* 3.2, pp. 258–268. DOI: 10.3390/automation3020013. (Visited on 06/29/2026).

Malburg, Lukas et al. (2021). "Object Detection for Smart Factory Processes by Machine Learning". In: *Procedia Computer Science* 184, pp. 581–588. DOI: 10.1016/j.procs.2021.04.009. (Visited on 06/29/2026).

Marks, Paul (2019). *Prototyping Faster with Laser Cutters*. URL: https://cacm.acm.org/news/prototyping-faster-with-laser-cutters/ (visited on 06/29/2026).

Mohammed, Amin A. and M. Sunar (2015). "Kinematics modeling of a 4-DOF robotic arm". In: *2015 International Conference on Control, Automation and Robotics*. DOI: 10.1109/iccar.2015.7166008. (Visited on 06/29/2026).

Protolabs (2024). *The Benefits of 3D Printing for Rapid Prototyping*. URL: https://www.protolabs.com/en-gb/resources/blog/the-benefits-of-3d-printing-for-rapid-prototyping/ (visited on 06/29/2026).

Salazar, Maxwell et al. (2024). “Impact of 3D printing technology for the construction of a prototype of low-cost robotic arm prostheses”. In: *Advances in Mechanical Engineering* 16.12. DOI: 10.1177/16878132241307065. (Visited on 06/29/2026).

Shen, Jie Lim and Irda Hassan (2014). “Design and Development of Colour Sorting Robot”. In: *Journal of Engineering Science and Technology EURECA*, pp. 71–81. URL: https://jestec.taylors.edu.my/eureca2014(1)_1_2015/eureca_14(1)_71_81.pdf (visited on 06/29/2026).

Singh, Hardial (2025). “Artificial Intelligence and Robotics Transforming Industries with Intelligent Automation Solutions”. In: *SSRN Electronic Journal*. DOI: 10.2139/ssrn.5267868. (Visited on 06/29/2026).

Skilton, Mark and Felix Hovsepian (2018). *The 4th Industrial Revolution*. Cham: Springer International Publishing. ISBN: 9783319624785. DOI: 10.1007/978-3-319-62479-2. (Visited on 06/29/2026).

Soori, Mohsen et al. (2024). “Intelligent robotic systems in Industry 4.0: A review”. In: *Journal of Advanced Manufacturing Science and Technology*. DOI: 10.51393/j.jamst.2024007. (Visited on 06/29/2026).

Surati, Sharath et al. (2021). “Pick and Place Robotic Arm: A Review Paper”. In: *International Research Journal of Engineering and Technology (IRJET)* 08.02. (Visited on 06/29/2026).

Trevelyan, James, William R. Hamel, and Sung-Chul Kang (2016). *Robotics in Hazardous Applications*. Springer Handbook of Robotics. DOI: 10.1007/978-3-319-32552-1_58. (Visited on 06/29/2026).

United Nations (2026). *The 17 Goals*. URL: https://sdgs.un.org/goals (visited on 08/26/2026).

Zou, Bipan et al. (2021). “Robotic Sorting Systems: Performance Estimation and Operating Policies Analysis”. In: *Transportation Science* 55.6. DOI: 10.1287/trsc.2021.1053. (Visited on 06/29/2026).

A. Mechanical Drawings

